UNIVERSITATEA BABEŞ-BOLYAI

Facultatea de Ştiinţe Economice şi Gestiunea Afacerilor

Informatică Economică

Lucrare de licenţă

Coordonator ştiinţific,

Lect. univ. dr. Florina **COVACI**

2025

UNIVERSITATEA BABEŞ-BOLYAI

Facultatea de Ştiinţe Economice şi Gestiunea Afacerilor

Informatică Economică

Lucrare de licenţă

„Certification Hub” – aplicație web pentru gestionarea cursurilor și certificarea candidaților, realizată cu ASP.NET Core, C# și Angular.

Coordonator ştiinţific,

Lect. univ. dr. Florina **COVACI**

2025

Rezumat

Lucrarea abordează tematica gestionării cursurilor online și a certificării competențelor utilizatorilor, prin dezvoltarea aplicației web Certification Hub. Aceasta permite personalului autorizat — experți sau specialiști din domenii precum economia, programarea și altele să creeze, editeze și publice cursuri, precum și să construiască examene asociate fiecărui modul educațional. Utilizatorii pot parcurge conținutul cursurilor, susține testări, iar în cazul promovării, pot obține certificate de competență. Aplicația este destinată atât celor care oferă instruire, cât și celor care doresc să acumuleze cunoștințe și să se certifice, precum programatori, studenți sau profesioniști în reconversie. Platforma oferă o interfață intuitivă, funcționalități academice și un flux bine structurat de învățare și validare a cunoștințelor. Din punct de vedere tehnic, soluția a fost implementată folosind ASP.NET Core (C#) pentru backend și Angular (TypeScript, HTML, SCSS) pentru frontend. Printre funcționalitățile dezvoltate se numără: gestionarea utilizatorilor și a permisiunilor, administrarea cursurilor și examenelor, trimiterea de emailuri automate și generarea certificatelor digitale în funcție de rezultatele obținute.

Cuprins

Abrevieri ..... **iv**
Lista tabelelor şi figurilor ..... **v**
Introducere **1**
1. Raportul de analiză **3**
1.1 Identificarea și descrierea problemei 3
1.1.1 Motivație 4
1.2 Context 6
1.2.1 Fațeta subiect 6
1.2.2 Fațeta utilizare 7
1.2.3 Fațeta IT 8
1.2.4 Fațeta dezvoltare 9
1.3 Cerințe de sistem 10
1.3.1 Surse de cerințe 11
1.3.2 Elicitația cerințelor 11
2. Proiectarea sistemului informatic **16**
2.1 Proiectarea logică 16
2.1.1 Arhitectura sistemului 18
2.1.2 Baza informațională 22
2.2 Proiectarea tehnică 23
2.2.1 Structura fizică a datelor 24
2.2.2 Procese 28
2.3 Tehnologii specifice 33
2.3.1 Limbajul C# 33
2.3.2 ASP.NET Core 33
2.3.3 Serviciul REST 33
2.3.4 Entity Framework (EF) Core 34
2.3.5 Docker 34
2.3.6 Redis 34
2.3.7 Angular 35
2.3.8 SCSS 35
2.3.8 PostgreSQL 35
2.3.8 MongoDB 35
3. Implementarea sistemului informatic **36**
3.1 Crearea tabelelor utilizând EF Core 36
3.2 Programarea asincronă 40
3.3 Gestiunea secretelor 41
3.4 Gestionarea Redis 47
3.5 Funcționalități de bază și remarcabile 47
3.6 Comunicarea cu serviciul web 58
4. Testarea sistemului informatic **62**
4.1 Testare manuală 62
Concluzii **66**
Bibliografie ..... Error! Bookmark not defined.

Abrevieri

API,	Application Programming Interface
IDE,	Integrated Development Environment
REST,	Representational State Transfer
HTTP,	Hypertext Transfer Protocol
SCSS,	Sassy CSS
JWT,	JSON Web Token
JSON,	JavaScript Object Notation
MVC,	Model View Controller
ORM,	Object-Relational Mapping
SQL,	Structured Query Language
URI,	Uniform Resource Identifier
UI,	User Interface
DB,	Database
TS,	Typescript

Lista tabelelor şi figurilor

Tabele:

Tabel 1. Prezentarea tipurilor de utilizatori, al rolurilor și obiectivele acestora 12
Tabel 2. Prezentarea cazurilor de utilizare comune utilizatorilor 13
Tabel 3. Prezentarea cazurilor de utilizare ale administratorilor 14
Tabel 4. Prezentarea cazurilor de utilizare ale candidaților 15

Figuri:

Figura 1. Diagrama Fishbone pentru a evidenția problemele platformelor de cursuri 4
Figura 2. Diagrama de descompunere a obiectivelor propuse 6
Figura 3. Structura aplicației 17
Figura 4. Arhitectura de tip client-server 18
Figura 5. Structura de tip Clean Arhitecture 19
Figura 6. Prezentarea pachetelor NuGet utilizabile în cadrul aplicației 20
Figura 7. Prezentarea secțiunii de acces a pachetelor NuGet în cadrul Rider IDE 21
Figura 8. Prezentarea container-ului Redis, în cadrul interfeței aplicației Docker 22
Figura 9. Diagrama ERD (Entity Relationship Diagram) realizată pentru UserManagement API 25
Figura 10. Diagrama ERD (Entity Relationship Diagram) realizată pentru Management API 26
Figura 11. Poză cu structura colecției ce reprezintă cursurile în MongoDB 27
Figura 12. Poză cu conținut de curs din MongoDB 28
Figura 13. Prezentarea unui mail de înregistreare pe platforma Certification Hub 29
Figura 14. Diagrama use-case ce evidențiază procesul de înregistrare și autentificare 30
Figura 15. Diagrama use-case ce evidențiază procesul de modificare a cursurilor 31
Figura 16. Prezentarea fișierului de secrete în cadrul aplicației 41
Figura 17. Prezentarea interfeței Swagger pentru UserManagement.Api 63
Figura 18. Prezentarea zonei de unde se extrage token-ul de acces pentru cereri 63
Figura 19. Prezentarea zonei de autentificare din cadrul Swagger UI 64
Figura 20. Cerere realizată în cadrul Swagger UI 64

Introducere

Motivul pentru care am ales această temă de licență este strâns legat de parcursul meu profesional ca dezvoltator software, cât și de experiențele personale acumulate în rolul de cursant. Încă din perioada primului internship am fost implicat în realizarea unei aplicații web cu tematică universitară, care gestiona mai multe aspecte educaționale, printre care și cursurile, examenele asociate și rezultatele obținute de studenți. În acea etapă, din lipsă de experiență și planificare, am întâmpinat dificultăți majore în ceea ce privește crearea și corelarea logică a cursurilor cu examenele, atât din perspectiva profesorului, cât și din cea a studentului. Provocările tehnice și funcționale întâmpinate atunci m-au motivat să revin asupra acelei componente, să o regândesc și să aplic cunoștințele dobândite în timp, obținând în prezent o soluție eficientă, dinamică și extensibilă. Totodată, din postura de cursant, am constatat că multe dintre cursurile de care aveam nevoie nu erau disponibile gratuit. De cele mai multe ori, am fost nevoit să caut informații din surse multiple, pierzând timp valoros și confruntându-mă cu lipsa unor materiale coerente și complete.

Aplicația este destinată atât profesorilor și experților în diverse domenii, cât și utilizatorilor interesați să acumuleze cunoștințe și să obțină certificări, precum studenți, programatori sau alți profesioniști. Platforma permite crearea de versiuni preliminare ale cursurilor, editabile în mod controlat, fără a afecta varianta activă. Această abordare de tip „versiune în lucru” permite testarea și ajustarea conținutului fără riscul pierderii datelor originale. Cursurile pot include elemente de structură variate – paragrafe, titluri, liste sau combinații ale acestora – iar pentru componenta de examinare sunt disponibile întrebări cu răspunsuri multiple, marcate drept corecte sau incorecte. Utilizatorii finali, respectiv personele doritoare pentru a învăța sau specializa pe anumite domenii, pot primi invitații pentru a accesa o multitudine de cursuri, a le parcurge, a susține testările aferente și, în funcție de rezultat, a obține sau nu o certificare în domeniul respectiv.

Pentru implementarea aplicației am adoptat o abordare pe mai multe microservicii, bine structurată. Pe partea de backend, am utilizat framework-ul ASP.NET Core, organizând codul în librării specializate: una pentru interacțiunea cu baza de date, alta pentru logica aplicației și altele pentru gestionarea utilizatorilor, invitațiilor, cursurilor și certificărilor. Pe partea de frontend, am dezvoltat aplicația în Angular, cu componente independente dedicate fiecărei zone funcționale: administrarea utilizatorilor, gestionarea cursurilor, susținerea

examenelor, precum și utilitare cum ar fi un serviciu global de erori și interceptori care adaugă automat cheia de autentificare în cereri.

Lucrarea de față este structurată în mai multe capitole, fiecare abordând o etapă distinctă în dezvoltarea aplicației Certification Hub. În **Capitolul 1** sunt prezentate contextul general, problematica identificată și justificarea alegerii temei, accentuând nevoia unei soluții informatice moderne pentru gestionarea cursurilor și certificarea utilizatorilor.

Capitolul 2 descrie proiectarea sistemului informatic, cu accent pe arhitectura logică și tehnică a aplicației. Se explică modul în care au fost structurate datele, componentele software utilizate și procesele esențiale care stau la baza funcționării sistemului.

În **Capitolul 3** este detaliată implementarea sistemului informatic, incluzând etapele tehnice de dezvoltare a aplicației, crearea tabelelor prin EF Core, programarea asincronă, gestionarea sesiunilor prin Redis, precum și comunicarea cu serviciile web. Sunt evidențiate funcționalitățile cheie și tehnologiile utilizate.

Capitolul 4 este dedicat testării sistemului informatic, fiind descris modul în care au fost verificate funcționalitățile aplicației prin testare manuală, pe baza scenariilor de utilizare. Scopul acestui proces a fost identificarea erorilor și asigurarea unei funcționări stabile a aplicației, atât la nivel de frontend, cât și în interacțiunea cu backend-ul.

1. Raportul de analiză

În raportul de analiză am ales să identific și să descriu problema care a stat la baza deciziei de a dezvolta un sistem informatic dedicat gestionării cursurilor, susținerii examinării și certificării competențelor dobândite. În acest capitol este prezentat contextul în care am ales să creez aplicația web Certification Hub, alături de cele patru fațete specifice care definesc arhitectura conceptuală: fațeta subiect, fațeta utilizare, fațeta IT și fațeta dezvoltare.

1.1 Identificarea și descrierea problemei

Majoritatea platformelor actuale specializate în oferirea de cursuri și certificări (de exemplu Courserea, Udemy) presupun costuri ridicate, inaccesibile pentru o parte importantă a utilizatorilor interesați de formare profesională sau personală. În plus, multe dintre aceste soluții sunt limitate în ceea ce privește posibilitatea de creare liberă și dinamică a conținutului, iar integrarea unui sistem eficient de examinare este adesea greoaie sau rigidă. Examinarea este tratată ca o etapă separată, lipsită de legătură directă cu materialul parcurs, ceea ce afectează coerența procesului educațional.

Totodata, conform unei reviste de specialitate, majoritatea platformelor intâmpină probleme precum abandonul cursurilor, din motive precum: lipsa de motivație sau dificultăți în a urmări conținutul cursurilor (BEZERRA & Terra da SILVA, 2017)

Mai jos voi prezenta o diagramă în care apar majoritatea problematicilor în cadrul platformelor asemănătoare, experimentate pe cont propriu și de asemenea experimentate de alte persoane pe care le-am întrebat:

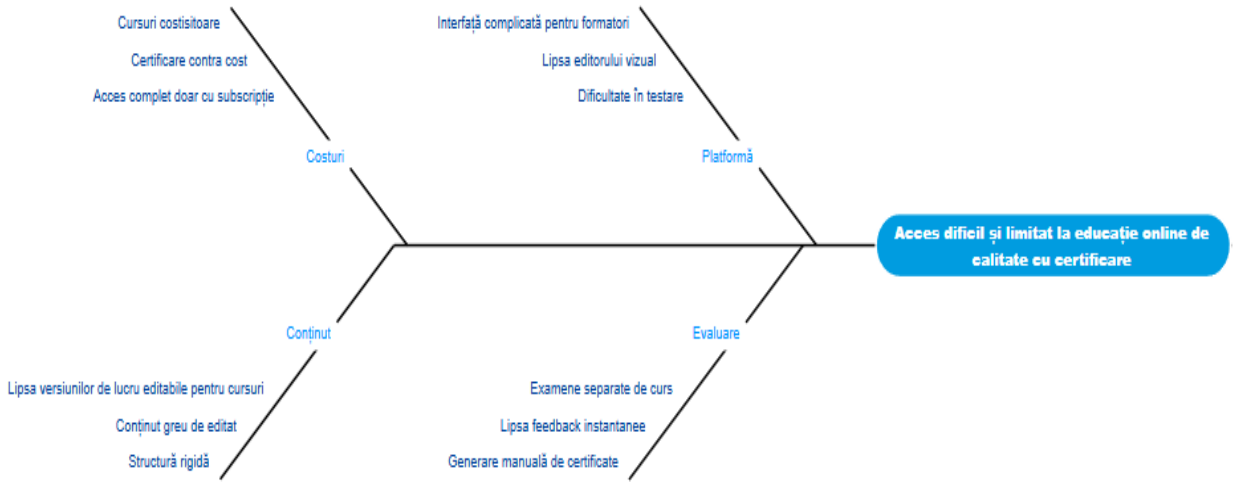


Figura 1. Diagrama Fishbone pentru a evidenția problemele platformelor de cursuri

Figura reprezintă o diagramă, în care se pot observa unele problematici ale platformelor pentru cursuri exitente, experimentate individual, dar și prin întrebarea altor persoane. Realizată în Visual Paradigm Smart Board, *Fishbone (Ishikawa) Diagram.* Regăsit *la* https://online.visual-paradigm.com/app/smart-board/#/?proj=0&gallery=/repository/98da929c-f711-401b-bea6-b8be37b15fea.xml&name=Fishbone%20(Ishikawa)%20Diagram&diagramType=FishboneDiagram

1.1.1 Motivație

Motivația personală care a stat la baza acestui proiect a fost dorința de a susține accesul liber și echitabil la educație. Consider că procesul de învățare ar trebui să fie unul accesibil tuturor, plăcut și lipsit de bariere financiare. Într-o perioadă în care educația de calitate este adesea condiționată de costuri ridicate, am simțit nevoia de a crea o platformă care să ofere oportunități de dezvoltare pentru oricine, indiferent de resursele materiale disponibile.

Privind din perspectiva unui formator, am identificat necesitatea unei soluții flexibile, care să permită crearea, modificarea și actualizarea conținutului educațional într-un mod simplu și intuitiv. În plus, am considerat esențial ca interacțiunea dintre candidat și examen să fie una prietenoasă, clară și lipsită de complicații tehnice, astfel încât accentul să rămână pe învățare și nu pe dificultăți de utilizare.

Obiectivul general al aplicației este dezvoltarea unei platforme web care să încurajeze formatorii, profesorii sau specialiștii din diverse domenii să creeze cursuri și să împărtășească aptitudini și informații cu persoane dornice de învățare, fie că sunt studenți, fie aflați la început de carieră. Totodată, aplicația urmărește să susțină procesul de formare prin testarea

cunoștințelor acumulate și generarea automată de certificări pe baza rezultatelor obținute. Un aspect esențial îl constituie și asigurarea securității datelor utilizatorilor, precum și gestionarea controlului accesului pe platformă printr-un sistem de permisiuni. Din obiectivul general se desprind mai multe obiective specifice, grupate în funcție de funcționalitățile majore ale aplicației:

1. Gestionarea înregistrării și autentificării utilizatorilor
    - Înregistrarea utilizatorilor pe baza unei invitații primite prin e-mail;
    - Crearea contului și autentificarea în platformă;
    - Securizarea contului prin parolă, conform unor standarde predefinite;
2. Administrarea utilizatorilor și a permisiunilor
    - Editarea permisiunilor unui utilizator în funcție de rolul său;
    - Ștergerea utilizatorilor din sistem, dacă este cazul.
3. Gestionarea invitațiilor
    - Adăugarea de adrese de e-mail către care pot fi trimise invitații în platformă;
    - Ștergerea invitațiilor existente;
    - Paginare și filtrare a listei de invitații.
4. Crearea și administrarea cursurilor
    - Crearea de cursuri noi, structurate pe versiuni;
    - Posibilitatea de a modifica un curs fără a afecta versiunea activă;
    - Salvarea modificărilor într-o versiune nouă sau înlocuirea celei existente;
    - Editarea dinamică a conținutului (paragrafe, titluri, liste etc.);
    - Adăugarea secțiunii de examinare, cu întrebări și variante de răspuns;
    - Ștergerea, filtrarea și paginarea cursurilor.
5. Examinarea și generarea certificatelor
    - Posibilitatea susținerii examenelor pe baza materialelor parcurse;
    - Acordarea certificatelor digitale după promovarea examenelor;
    - Posibilitatea descărcării diplomelor generate;
    - Păstrarea unui istoric al examenelor promovate de către fiecare student;
    - Acces permanent la diplomele obținute;
    - Paginarea și filtrarea listelor de examene sau certificări.

Mai jos voi prezenta o diagramă, sintetizată și structurată cu obiectivele menționate mai sus:

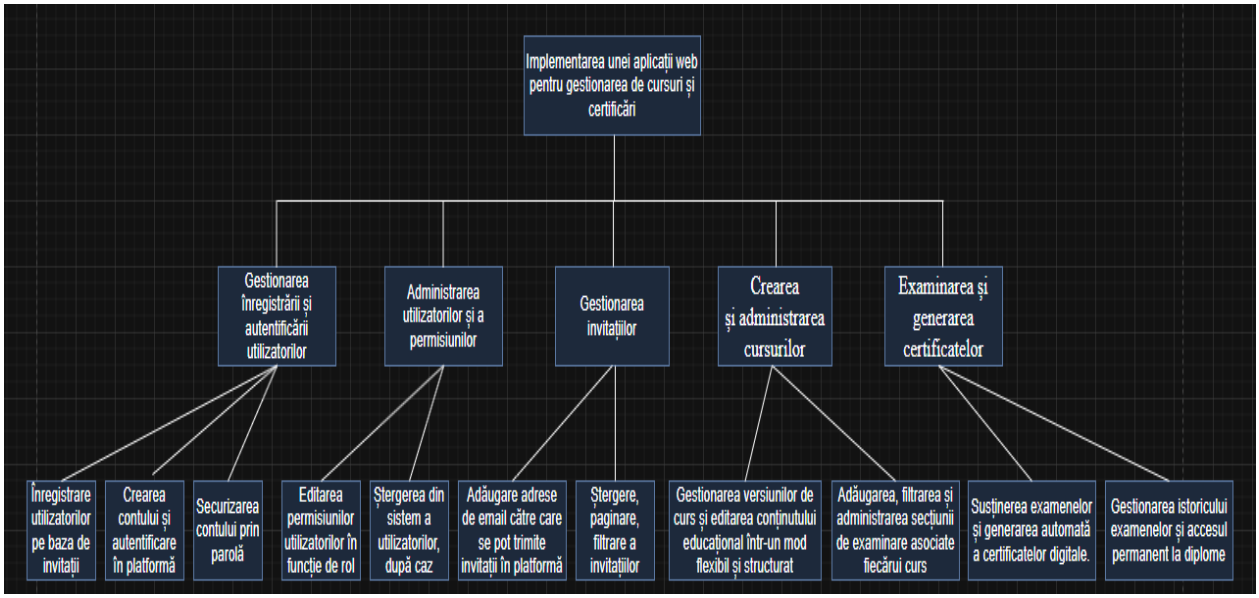


Figura 2. Diagrama de descompunere a obiectivelor propuse

Figura reprezintă o diagramă, cu obiectivele propuse în cadrul realizării aplicației, creată cu ajutorul site-ului draw.io, regăsit la: https://app.diagrams.net/

1.2 Context

În acest subcapitol este prezentat contextul în care a fost dezvoltată soluția propusă. Descrierea se realizează folosind metodologia fațetelor, care presupune analizarea sistemului informatic din patru perspective distincte. Prima este fațeta subiect, care tratează evenimentele și elementele relevante din domeniul în care sistemul își desfășoară activitatea. A doua este fațeta utilizare, unde sunt prezentate modurile în care utilizatorii interacționează cu aplicația și beneficiile acestora. A treia, fațeta IT, face referire la tehnologiile software și infrastructura folosită pentru dezvoltarea sistemului. În final, fațeta dezvoltare acoperă aspectele legate de procesul de dezvoltare propriu-zis și factorii care pot influența evoluția ulterioară a aplicației.

1.2.1 Fațeta subiect

Aplicația Certification Hub abordează domeniul educației și al formării profesionale, într-un context în care digitalizarea procesului de învățare a devenit esențială. Aceasta își propune să rezolve o problemă des întâlnită în platformele actuale: lipsa unei soluții complete, gratuite și

coerente care să reunească într-un singur loc livrarea conținutului educațional, evaluarea cunoștințelor și generarea de certificări digitale. Multe platforme de genul oferă aceste servicii în mod fragmentat sau condiționat de costuri, ceea ce limitează accesul pentru o mare parte din utilizatori.

Ideea de a dezvolta această aplicație a pornit dintr-o experiență personală frustrantă, în care am început mai multe cursuri online doar pentru a descoperi că accesul complet la material sau la evaluări era posibil doar în baza unei plăți. De multe ori, exact în momentul în care conținutul devenea clar și util, utilizatorului i se solicita o subscripție. Această barieră financiară, combinată cu lipsa unor platforme bine integrate, a dus la inițiativa de a crea un spațiu educațional gratuit, complet și accesibil tuturor.

În cadrul aplicației, cursurile sunt compuse din elemente de tip paragraf, titlu, cod sursă, liste simple sau combinate, oferind flexibilitate în structurarea conținutului. Examinarea este unică pentru fiecare curs și constă într-o secțiune în care administratorul poate introduce un set variat de întrebări, fiecare având una sau mai multe variante de răspuns. După completarea testului, punctajul este calculat automat, iar în funcție de rezultat, se generează o certificare digitală personalizată.

1.2.2 Fațeta utilizare

Aplicația este concepută pentru a deservi două tipuri principale de utilizatori: administratori și candidați. Fiecare categorie beneficiază de funcționalități adaptate rolului său, cu scopul de a asigura o utilizare clară, sigură și eficientă a platformei.

Administratorii sunt responsabili de gestionarea conținutului și a utilizatorilor din aplicație. Ei dețin permisiuni de citire, scriere, modificare și ștergere în mai multe zone esențiale: gestionarea conturilor de utilizator, gestionarea invitațiilor, gestionarea cursurilor și a examinării. Un administrator poate, de exemplu:

- să creeze sau să editeze cursuri educaționale folosind structuri variate de conținut (paragrafe, titluri, liste, cod etc.);
- să formuleze seturi de întrebări pentru fiecare curs, cu răspunsuri grilă și corecturi automate;
- să trimită invitații către viitori utilizatori prin introducerea adreselor de e-mail, sistemul generând un link personalizat pentru înregistrare;
- să atribuie sau să modifice permisiunile altor administratori ori candidați;

- să filtreze și să sorteze listele de cursuri, utilizatori și invitații, în funcție de criterii relevante;
- să șteargă definitiv conținutul sau utilizatorii inactivi, menținând astfel platforma curată și organizată.

Pe de altă parte, candidații reprezintă utilizatorii finali ai aplicației. Aceștia nu pot modifica conținutul platformei, ci doar interacționează cu materialele educaționale puse la dispoziție de administratori. Candidații au posibilitatea de:

- a accesa și parcurge cursurile disponibile în interfața lor personalizată;
- a susține examene aferente fiecărui curs, care sunt configurate anterior de administratori;
- a primi, la finalul testului, un feedback imediat privind răspunsurile corecte și greșite;
- a vizualiza examenele promovate și, în urma acestora, a descărca certificările obținute în format digital.

Certificările sunt afișate într-o secțiune dedicată a platformei, filtrate automat astfel încât să apară doar acelea aferente testelor finalizate cu succes. Acest mecanism încurajează implicarea reală a candidatului și oferă o recompensă concretă pentru efortul depus.

Pentru a putea utiliza platforma, fiecare utilizator trebuie să se autentifice. Procesul de autentificare presupune completarea unui formular cu e-mail și parolă, iar sistemul validează formatul adresei de e-mail și impune un minim de caractere pentru parolă. Astfel, se asigură protecția datelor și prevenirea accesului neautorizat.

Interacțiunea este intuitivă, iar fiecare rol este asociat cu o interfață specifică. Utilizatorii beneficiază de un meniu clar, care le permite accesul rapid către funcțiile de interes. În plus, aplicația este concepută să minimizeze erorile și confuziile printr-un sistem de permisiuni bine definit și un control riguros al acțiunilor disponibile fiecărui utilizator.

1.2.3 Fațeta IT

Din punct de vedere tehnologic, aplicația este construită folosind o arhitectură de tip client–server, specifică sistemelor web moderne.

Partea de backend este implementată în ASP.NET Core, folosind limbajul C#, și oferă un set de API-uri prin care se gestionează funcționalitățile esențiale ale aplicației, cum ar fi

autentificarea utilizatorilor, administrarea cursurilor, procesul de examinare și generarea certificatelor.

Interfața aplicației este dezvoltată în Angular, utilizând TypeScript, SCSS și HTML, ceea ce permite o organizare modulară a componentelor și o interacțiune fluidă cu utilizatorul. Frontend-ul comunică în permanență cu backend-ul prin cereri HTTP, iar autentificarea este gestionată printr-un sistem bazat pe tokenuri (JWT), care asigură securizarea accesului în funcție de rolul utilizatorului.

Arhitectura este una bazată pe mai multe microservicii, ceea ce permite extinderea facilă a aplicației pe viitor iar datele utilizatorilor, ale cursurilor și ale examenelor sunt gestionate într-un mod centralizat și sigur, cu validări aplicate atât pe partea de client, cât și pe server.

În plus, aplicația implementează mecanisme de filtrare și paginare pentru a optimiza accesul la resurse.

1.2.4 Fațeta dezvoltare

Dezvoltarea aplicației a fost realizată într-un mod progresiv și organizat, fără a urma în mod formal o metodologie de dezvoltare software dedicată. Cu toate acestea, abordarea adoptată poate fi apropiată de principiile metodologiei Agile, în care procesul de construire a aplicației s-a desfășurat iterativ, cu planificări zilnice și ajustări rapide în funcție de nevoile observate pe parcurs.

Proiectul a fost realizat în întregime individual, ceea ce a implicat atât responsabilitățile de design și implementare, cât și testarea și optimizarea fiecărei funcționalități. Am lucrat constant, în special în weekenduri, dedicând întreaga zi etapelor de dezvoltare, dar și în timpul săptămânii, în funcție de disponibilitate. În fiecare zi îmi stabileam obiective clare, de exemplu, dezvoltarea unei secțiuni complete precum gestionarea cursurilor, a sistemului de permisiuni sau a examinării. Atunci când finalizam mai repede decât estimasem, profitam de timp pentru a repara erori, a îmbunătăți stilizarea interfeței sau a adăuga funcționalități suplimentare utile.

Pe parcursul dezvoltării au existat și provocări. Una dintre cele mai importante decizii a fost refacerea completă a frontend-ului, deoarece versiunea inițială era simplistă, rigidă și vizual învechită. Am optat pentru o modernizare a interfeței, axându-mă pe o experiență de utilizare mai fluidă, componente mai bine organizate și o estetică adaptată cerințelor aplicațiilor moderne.

Aplicația a fost construită într-un mod care dispune de mai multe microservicii, astfel încât fiecare secțiune să poată fi dezvoltată și menținută independent. Acest lucru a oferit un avantaj semnificativ în procesul de testare și extindere, deoarece fiecare zonă are o logică proprie clară și separată.

Privind în perspectivă, există numeroase direcții de extindere și îmbunătățire a aplicației. Printre planurile viitoare se numără:

- adăugarea unor opțiuni avansate de filtrare a accesului la cursuri, astfel încât un administrator să poată decide care candidați pot accesa un anumit material educațional;
- implementarea unui sistem mai strict de validare a structurii cursurilor, pentru a evita erorile logice și organizatorice;
- extinderea conținutului posibil al cursurilor, prin adăugarea suportului pentru materiale video, documente PDF, imagini și alte resurse multimedia;
- eventual, dezvoltarea unei zone de raportare și analiză, care să ofere instructorilor informații despre progresul și performanța candidaților.

Această experiență de dezvoltare, desfășurată într-un ritm susținut, m-a ajutat nu doar să consolidez cunoștințele tehnice, ci și să-mi dezvolt abilități de organizare, planificare și adaptare în fața provocărilor reale care apar în procesul de construire a unei aplicații complete.

1.3 Cerințe de sistem

În această secțiune sunt prezentate cerințele care au stat la baza dezvoltării aplicației Certification Hub. Pentru a construi un sistem informatic eficient și adaptat nevoilor utilizatorilor, este esențială înțelegerea corectă a cerințelor funcționale și nefuncționale ale aplicației, precum și a contextului în care acestea au fost formulate.
Secțiunea este structurată în mai multe subcapitole, care detaliază:

- sursele de cerințe, adică persoanele sau situațiile reale care au generat nevoia de dezvoltare a sistemului;
- procesul de elicitație, care descrie metodele prin care au fost extrase și definite cerințele aplicației, în funcție de nevoile identificate ale utilizatorilor;

- cerințele funcționale și nefuncționale, care vor fi descrise în termeni clari, pentru a evidenția comportamentul așteptat al aplicației și constrângerile tehnice care trebuie respectate.

1.3.1 Surse de cerințe

Cerințele care au stat la baza dezvoltării aplicației au fost extrase dintr-o combinație de experiențe personale directe, observații asupra modului actual în care sunt oferite cursurile și certificările online, precum și din analiza limitărilor platformelor existente. Sursa principală de cerințe o reprezintă utilizatorii finali ai platformei, grupați în două categorii distincte:

- Formatori / administratori, adică persoane care doresc să creeze cursuri educaționale, să structureze conținut relevant și să ofere participanților posibilitatea de a fi evaluați și certificați;
- Cursanți / candidați, adică persoane interesate să parcurgă materiale de învățare, să susțină examene și să obțină certificări care să le valideze cunoștințele și competențele.

Un alt factor important l-a reprezentat lipsa accesului gratuit și integrat la materiale educaționale de calitate. În contextul în care multe platforme de asemănătoare impun costuri considerabile sau oferă funcționalitățile în mod fragmentat, a fost identificată nevoia unei soluții care să elimine aceste bariere. Certification Hub răspunde, astfel, nevoii de acces liber, complet și bine organizat la învățare și certificare.

De asemenea, cerințele au fost influențate de tendințele actuale din domeniul educației digitale, în care utilizatorii se așteaptă la platforme ușor de utilizat, intuitive și accesibile de oriunde. Sistemul a fost gândit pentru a răspunde atât nevoilor de organizare ale formatorilor, cât și nevoii de simplitate și claritate în procesul de învățare al cursanților.

1.3.2 Elicitația cerințelor

Pentru identificarea nevoilor reale ale utilizatorilor și extragerea cerințelor funcționale, am folosit două metode complementare: autoanaliza experienței directe și brainstorming-ul bazat pe exemple și scenarii reale.

Fiind atât dezvoltatorul cât și inițiatorul proiectului, am pornit de la o analiză critică a platformelor pe care le-am utilizat anterior în procesul de învățare personală. Această experiență a scos în evidență probleme precum: lipsa de coerență între curs și examen, dificultăți de navigare, acces limitat fără plată și lipsa feedback-ului relevant.

În cadrul sesiunilor de brainstorming, am simulat scenarii de utilizare din ambele perspective: administrator și candidat. Acest exercițiu m-a ajutat să conturez fluxuri clare și să stabilesc cerințele prioritare. De exemplu, pentru administratori a fost esențial să poată crea cursuri ușor, să controleze accesul la conținut și să definească examene personalizate. Pentru candidați, am prioritizat accesibilitatea, claritatea interfeței și existența unui sistem de feedback automat și certificare digitală.

Un alt aspect important în definirea cerințelor a fost identificarea potențialelor riscuri. Printre acestea se numără:

- accesul neautorizat la anumite funcționalități (rezolvat prin sistemul de permisiuni);
- lipsa unui feedback clar în urma examenelor (rezolvat prin evaluare automată);
- confuzia legată de conținut sau parcurgerea cursurilor (rezolvat prin structurare modulară și versiuni „în lucru”).

Mai jos voi reprezenta într-un tabel, tipul de utilizatori, rolurile acestora și obiectivele principale:

Tabel 1. Prezentarea tipurilor de utilizatori, al rolurilor și obiectivele acestora

Actor	*Roluri*	*Obiective principale*
Administrator	- Creare, editare și ștergere cursuri și examene; - Gestionare utilizatori și permisiuni; - Trimitere invitații prin email; - Vizualizare rezultate examene; - Generare certificări.	- Oferirea de conținut educațional de calitate; - Evaluarea candidaților; - Administrarea structurată a platformei.
Candidat	- Acceptarea invitației și înregistrare în platformă; - Vizualizare și parcurgere cursuri; - Susținere examen; - Vizualizare feedback și	- Acces gratuit la învățare; - Dobândirea de competențe; - Obținerea certificatelor digitale.

certificare;
- Descărcare diplomă.

Pentru o înțelegere mai profundă a funcționalităților sistemului informatic, este important să fie analizate cazurile de utilizare care definesc modul în care utilizatorii interacționează cu aplicația. Aceste cazuri evidențiază acțiunile posibile și existente. Prin identificarea clară a acestor interacțiuni, se pot urmări mai ușor cerințele sistemului, scenariile tipice de utilizare și punctele cheie în procesul de dezvoltare.

Mai jos voi reprezenta într-un alt tabel, o descriere amănunțită a cazurilor de utilizare ale sistemului informatic prezentat, comune administratorilor și candidaților

Tabel 2. Prezentarea cazurilor de utilizare comune utilizatorilor

Număr	*Caz de utilizare*	*Descriere*
1.	Autentificare	În momentul accesării aplicației web de către un utilizator, acesta întâmpină interfața de autentificare, și anume o pagina care dispune de un formular pentru completarea credențialelor precum: e-mail și parola, și un buton de submit, pentru a porni procesul de autentificare pe baza informațiilor introduse.
2.	Înregistrare	În urma unei invitații create de administratori, un mail se va trimite la adresa de e-mail introdusă de acesta, iar utilizatorul țintă va putea accesa aplicația prin intermediul unui link pe care mail-ul îl dispune. Acest mail diferă in funcție de tipul de invitație trimisă. În urma accesării link-ului de pe mail, utilizatorul va fi redirecționat pe o pagina în care se prezintă un formular pentru completarea datelor precum: nume, prenume, parola și repetarea parolei, precum și un buton de submit, pentru a porni procesul de înregistrare în cadrul aplicației.
3.	Vizionare dashboard	Prima pagină pe care utilizatorul este redirecționat în urma autentificării este cea de dashboard. Aceasta pagină conține rapoarte sau informații utile

		utilizatorului (dar difera în funcție de rol), de exemplu un administrator poate vedea numarul de utilizatori activi sau inactivi și numărul de invitații active, acceptate sau expirate.
4.	Vizionare profil	Aplicația dispune de o zonă de profil în care utilizatorii își pot vizualiza date precum: numele, prenumele, e-mail-ul și rolul acestora în aplicație (administrator sau candidat).
5.	Vizionare cursuri	Aplicația dispune de o zona de cursuri principală, de unde se pot realiza filtrări în funcție de nume, status sau etichete. De asemenea dispune de o paginare corespunzătoare (dar de regulă, se afișează un numar maxim de 5 cursuri per pagină). Din intermediul acestei pagini, la un click distanță asupra unui card ce conține informații de baza ale unui curs specific, utilizatorul va fi redirecționat în zona de detalii ale cursului. În funcție de tipul utilizatorului, înafara de detalii, se pot observa zona de secțiuni și examinare ale cursului. Inițial zona de detalii și secțiuni este vizibilă pentru candidați, iar restul pentru administratori, deoarece zonele acestea pot fi modificate.

Mai jos voi prezenta câte un tabel, pentru descrierea amănunțită a cazurilor de utilizare ale sistemului informatic prezentat, în functie de rolul utilizatorului:

Tabel 3. Prezentarea cazurilor de utilizare ale administratorilor

Număr	*Caz de utilizare*	*Descriere*
1.	Vizionare și accesare utilizatori	Administratorii cu suficiente permisiuni asupra acestei zone, pot filtra, sorta, edita și șterge utilizatori. Zona de editare a utilizatorilor, dispune de un modal în care apar toate permisiunile prezente ale acestuia, acestea pot fi

		modificate de către administrator pentru a reduce sau a adăuga access în cadrul aplicației.
2.	Vizionare și accesare cursuri	Administratorii cu suficiente permisiuni asupra cursurilor, pot accessa secțiunile și zona de examinare într-un mod diferit. Pot crea variante în lucru ale cursurilor, asupra cărora pot adăuga, șterge și edita conținut, fără a afecta varianta inițială a cursului. Toate schimbările de structură se reflectă în varianta inițială, doar în momentul în care se salvează modificările, astfel, varianta în lucru devenind variantă finală.
3.	Vizionare și accesare invitații	Administratorii cu suficiente permisiuni asupra acestei zone, pot filtra, sorta, crea și șterge invitații. Zona de creare a invitațiilor, presupune o modalitate prin care administratorul poate să introducă una sau mai multe adrese de e-mail, care sunt validate înaintea salvării. Pentru fiecare adresă, se va trimite un mail cu invitație specifică, în cadrul aplicației.

Tabel 4. Prezentarea cazurilor de utilizare ale candidaților

Număr	*Caz de utilizare*	*Descriere*
1.	Examinare	În urma vizualizării tuturor secțiunilor unui curs, unui candidat i se oferă posibilitatea de a trece printr-un proces de examinare. În momentul accesării exminării, candidatul este redirecționat către o pagina care dispune de un cronometru (cât durează examinarea), întrebarile legate de cursul vizionat și răspunsurile care pot avea una sau mai multe variante corecte de răspuns. Candidatul are libertate de a alege răspunsurile, iar la final de examinare, sistemul automat de feedback, coloreaza răspunsurile corecte cu verde, iar cele greșite cu roșu, afișând totodată punctajul obținut la fiecare întrebare și nota finală. Fiecare curs dispune de un număr de încercări ale procesului de examinare, dacă acesta este depășit,

		candidatul nu v-a mai putea trece prin acest proces, doar la intervenția unui administrator, cu access asupra aplicației.
2.	Vizionare și accesare certificări	În urma unei examinări, candidatul poate accesa o zonă specifică lui, și anume zona de certificări, unde sunt afișate toate examenele luate cu success. Aici candidatul poate selecta o certificare și instala o diplomă pe baza acesteia.

2. Proiectarea sistemului informatic

În acest capitol sunt prezentate etapele și componentele esențiale care au stat la baza dezvoltării aplicației web ,,Certification Hub”. Proiectarea sistemului informatic implică atât aspecte logice, cum ar fi arhitectura generală și modelul informațional, cât și aspecte tehnice legate de stocarea datelor, fluxurile de proces și structura fizică a elementelor ce compun aplicația. Această etapă a fost fundamentală pentru a asigura o implementare coerentă și scalabilă.

2.1 Proiectarea logică

Această secțiune evidențiază structura conceptuală a aplicației ,,Certification Hub”, concentrându-se pe modul în care sunt organizate și relaționate componentele sistemului. Vom prezenta arhitectura generală a aplicației și modul de comunicare între module. Proiectarea logică stă la baza tuturor deciziilor de implementare și reflectă modul în care aplicația răspunde cerințelor funcționale identificate în fazele anterioare.

Mai jos voi prezenta o diagramă Archimate în care explic modul în care e structurată aplicația. Din aceasta reies următoarele afirmații: Aplicația conține un backend alcătuit din două API-uri (UserManagement si Management). UserManagement API gestionează utilizatori, permisiuni și invitații, se folosește de o baza de date PostgreSQL și o instanță de Redis creată în Docker. Management API gestionează cursuri, examinări și certificări, se

folosește de o baza de date PostgreSQL și una de MongoDB (pentru gestionarea dinamică a cursurilor).

Pe lângă API-urile de mai sus, s-au construit librării: Core, Infrastructure, Repository (individuale) și UserManagementLibrary (realizat de UserManagementApi): UserManagementApi se folosește de librăriile Core, Infrastructure și Repository, iar ManagementApi se folosește de toate. Librăriile reprezintă niște pachete nugget private, salvate în Github.

Comunicarea Frontend și Backend se realizează prin serviciul REST, cu ajutorul controalelor expuse de fiecare API.

Aplicația rulează pe browser și e potrivită pentru laptopuri, calculatoare și folosește pentru Frontend HTML, SCSS, TS.

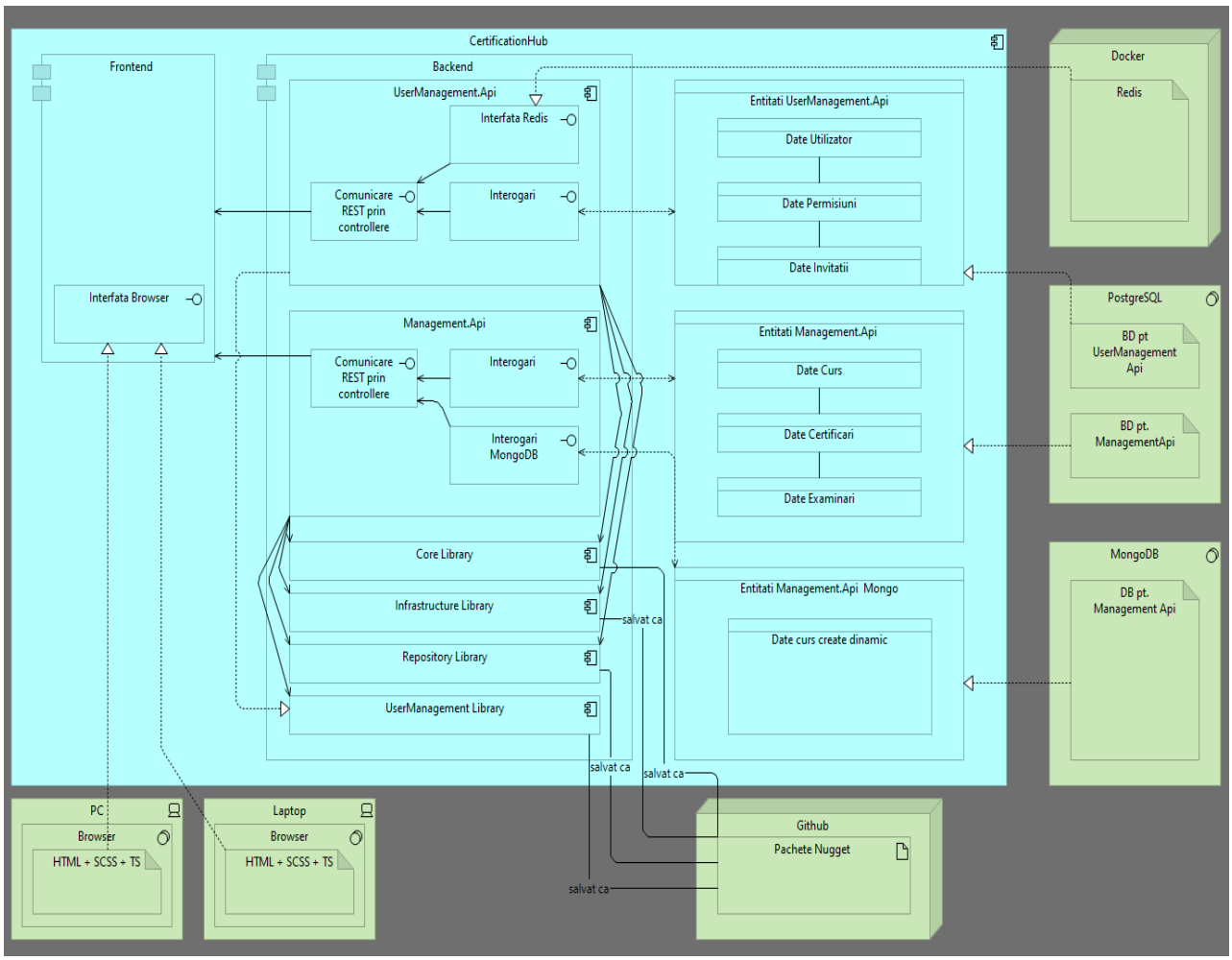


Figura 3. Structura aplicației

Figura reprezintă o diagramă realizată în Archimate, în care se prezintă structura aplicației

2.1.1 Arhitectura sistemului

Alegerea arhitecturii utilizate în dezvoltarea sistemului reprezintă un factor esențial în definirea stabilității și scalabilității aplicației. O arhitectură bine aleasă nu doar că optimizează procesul de dezvoltare, ci influențează în mod direct performanța, ușurința în mentenanță și posibilitățile de extindere ulterioară. Fiecare tip de arhitectură vine cu propriile avantaje și dezavantaje, motiv pentru care selecția acesteia trebuie realizată în raport cu cerințele concrete ale sistemului și cu scopurile urmărite.

În cazul aplicației, arhitectura adoptată este de tip client-server, o alegere potrivită având în vedere separarea clară între interfața cu utilizatorul și logica de procesare a datelor. Acest model permite ca partea de frontend, responsabilă cu interacțiunea utilizatorului, să funcționeze independent de logica aplicativă de pe server, ceea ce contribuie la o mai bună organizare a codului și la posibilitatea dezvoltării separate a celor două componente.
Acest tip de arhitectură se poate defini ca fiind ,,o arhitectură software în care clienții trimit constant cereri, iar serverul răspunde acestor cereri" fiecare având roluri diferite și acționând independent, presupunând un schimb bidirecțional de date. De regulă această arhitectură presupune 3 elemente esențiale: serverul, clientul și baza de date, dar pot exista și intermediare (Sulyman, 2014)

Mai jos voi include o diagramă, reprezentativă acestui tip de arhitectura:

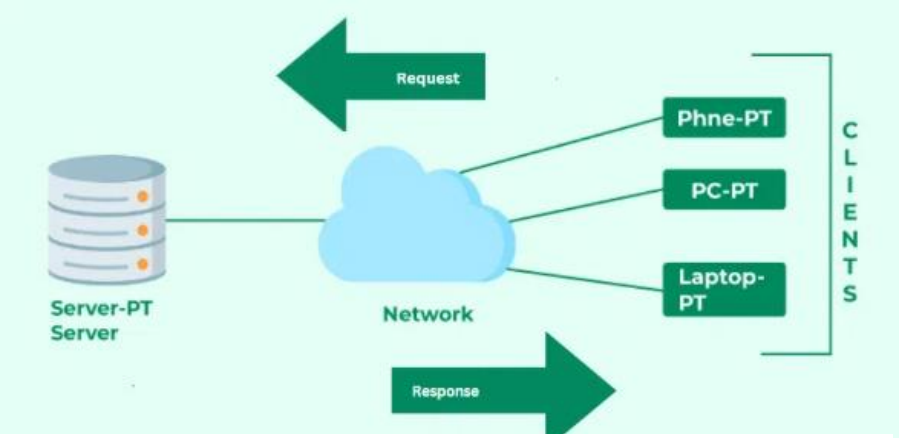


Figura 4. Arhitectura de tip client-server

Figura reprezintă interacțiunea bidirecțională dintre client și server, astfel: clienții fac cereri la server, serverul le procesează, apoi trimite câte un răspuns înapoi. Sursa *GeeksforGeeks*, *Client-Server Model*, secțiunea *How Does the Client-Server Model Work?* (GeeksforGeeks, 2025)

Structura backend-ului urmează principiile ,,Clean Architecture", ceea ce permite o separare clară între nivelurile de logică, interfață cu utilizatorul și infrastructură. Această structură

plasează logica de business și modelul aplicației în centrul structurii software, devenind astfel nucleul aplicației, iar din punct de vedere al dependințelor, în loc ca această logică sa fie dependentă de accesul la date sau alte detalii de infrastructura, relația este inversată: infrastructura și accesul la date devin dependente de nucleu (Microsoft, 2023)

Am ales această structură deoarece îmi permite să gestionez proiectul într-un mod clar și modular, definind un strat ,,Core", un strat ,,Infrastructure" și un strat ,,Presentation". Stratul ,,Core" conține logica de business, interfețele și implementările fundamentale. Stratul ,,Infrastructure" gestionează migrațiile și configurările contextului de date (pentru configurarea cu Entity Framework Core). Stratul ,,Presentation" conține fișierele ,,Program.cs" și ,,Startup.cs", responsabile de configurarea serviciilor și a middleware-urilor necesare pentru rularea aplicației, precum și clasele de tip ,,Controller", care expun funcționalitățile aplicației.

Mai jos voi prezenta o imagine reprezentativă ale acestei structuri:

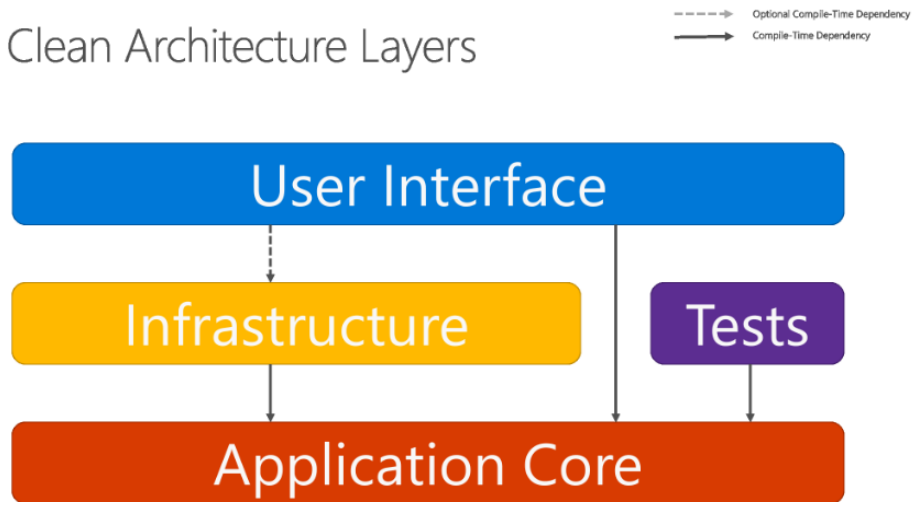


Figura 5. Structura de tip Clean Arhitecture

Figura reprezintă clar cum interfața utilizatorului, infrastructura și componentele de testare, toate depind de nucleul aplicației. Sursa *Microsoft Docs*, *Common web application architectures*, secțiunea *Clean Architecture*. (Microsoft, 2023)

Aplicația este structurată pe mai multe microservicii, fiind compusă din mai multe componente specializate. La nivel de Backend, sistemul include două API-uri distincte: unul dedicat gestionării cursurilor și proceselor de examinare, iar celălalt responsabil de gestionarea utilizatorilor, autentificării și permisiunilor. Ambele API-uri sunt dezvoltate în cadrul platformei ASP.NET Core și sunt completate de trei librării externe, publicate sub formă de pachete NuGet private, găzduite pe un repository GitHub. De menționat faptul că,

unele API-uri, de asemenea dispun o librărie externă, cu funcționalități necesare pentru alte API-uri (de exemplu, UserManagement.Api, expune o librărie ce conține elemente necesare pentru securizarea de endpoint-uri și o funcționalitate de autorizare).

Partea de frontend este realizată în Angular, fiind responsabilă cu interfața vizuală și interacțiunea utilizatorului cu aplicația. Aceasta este organizată pe componente independente, fiecare cu funcționalitate proprie, respectând bunele practici de separare a responsabilităților. Pentru gestionarea sesiunilor de autentificare, aplicația utilizează un container Docker cu o instanță de Redis, care permite stocarea temporară și rapidă a cheilor de acces. De asemenea, în cadrul soluției sunt integrate două baze de date: una relațională (PostgreSQL), utilizată pentru gestionarea principalelor entități ale aplicației, și una non-relațională (MongoDB), utilizată pentru stocarea datelor nestructurate sau dinamice. Această combinație de tehnologii oferă flexibilitate, scalabilitate și un timp de răspuns optim în funcție de natura datelor prelucrate.

Așa cum am menționat mai sus, pentru o mai bună organizare și reutilizare a codului, am ales să structurez logica de bază a aplicației sub forma unor librării externe publicate ca pachete NuGet, pe platforma Github. Am urmat această logică, deoarece, conform documentației Microsoft, ,,un instrument necesar oricărei platforme moderne îl reprezintă un mecanism prin intermediul căruia codul reutilizabil poate fi împachetat, distribuit și consumat eficient în alte proiecte, rol îndeplinit de NuGet, care reprezintă o arhivă cu extensia .nupkg și conține cod compilat sub formă de fișiere DLL, auxiliare și informații despre versiunea curentă, o descriere și dependințele pachetului" (Microsoft, 2022). Cu ajutorul acestor pachete, în intermediul celor două API-uri am putut folosi cod comun, lucru ce nu ar fi fost posibil fără aceste pachete, lipsa acestora ducând la creare de cod duplicat, cu aceeași funționalitate.

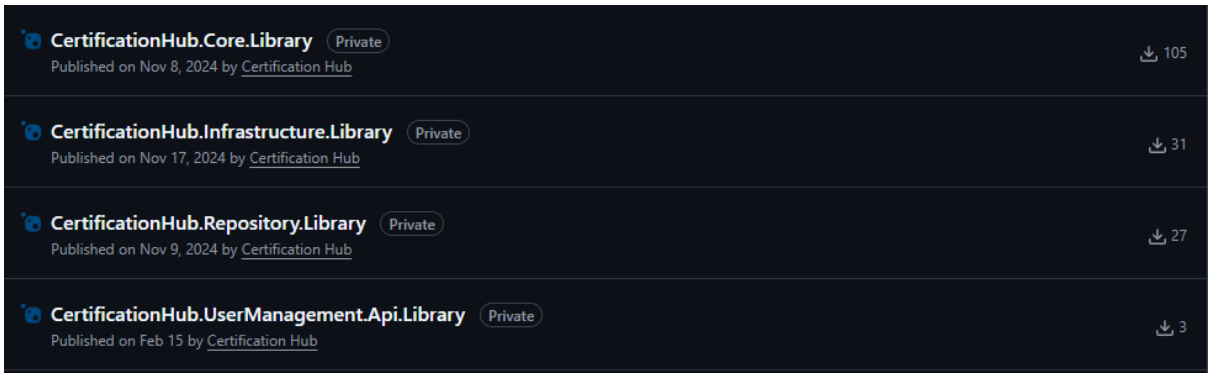


Figura 6. Prezentarea pachetelor NuGet utilizabile în cadrul aplicației

Figura reprezintă toate librăriile externe, folosite pentru a crea aplicația, evidențiindu-se astfel: trei librării principale Core, Infrastructure, Repository, și o librărie externă, dispusă chiar de API-ul UserManagement (care conține date specifice autorizării de endpoint-uri în cadrul aplicației).

Pentru utilizarea unui pachet, din cadrul unui API / Librărie sau orice alt tip de proiect, se va accesa zona de NuGet (de menționat faptul că, pe post de IDE am folosit aplicașia Rider de la JetBrains), se alege versiunea dorită și apoi se descarcă în unele dintre fișierele dorite din cadrul unui proiect.

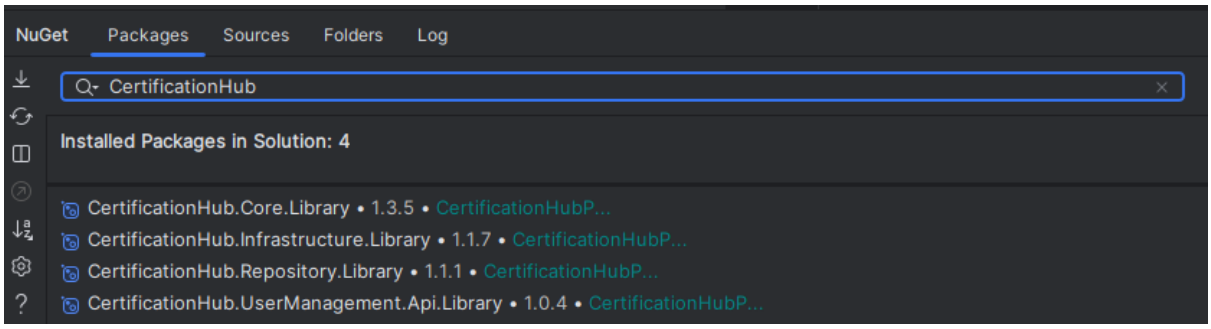


Figura 7. Prezentarea secțiunii de acces a pachetelor NuGet în cadrul Rider IDE

Figura reprezintă toate librăriile externe, create în cadrul aplicației Certification Hub și versiunile lor aferente.

Un ultim element tehnologic utilizat în cadrul aplicației este platforma Docker, care a fost integrată pentru a asigura un mediu de rulare standardizat și ușor de gestionat. Am utilizat un container dedicat unei instanțe de Redis, folosită pentru gestionarea sesiunilor de autentificare. Mai exact, la autentificarea unui utilizator, sistemul generează o cheie de access, iar identificatorul acesteia este stocat temporar în Redis, ceea ce permite o verificare rapidă și eficientă a validității sesiunii la fiecare cerere ulterioară. Acest mecanism contribuie la performanță, dar și la securitatea procesului de autentificare, reducând nevoia de interogări constante asupra bazei de date relaționale.

Containerul Redis este o componentă esențială pentru funcționarea corectă a aplicației și este utilizat în principal de API-ul UserManagement, responsabil de gestionarea conturilor, autentificare, permisiuni și sesiuni active. Mai multe detalii vor fi prezentate în cadrul subcapitolului dedicat technologiilor specifice.

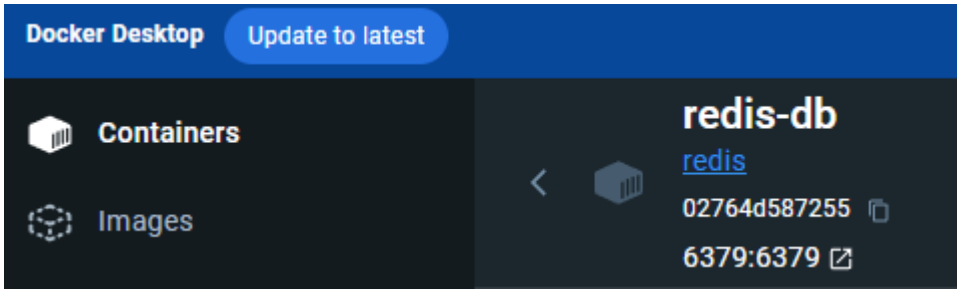


Figura 8. Prezentarea container-ului Redis, în cadrul interfeței aplicației Docker

În figura de mai sus, se observă un container denumit redis-db, care rulează pe portul 6379.

În cadrul aplicației, pentru partea de Backend, codul este structurat în jurul conceptului de controlere specializate, fiecare dintre acestea expunând endpoint-uri REST prin metode HTTP precum GET, POST, PATCH și DELETE.

Spre deosebire de arhitectura tradițională MVC (Model-View-Controller), folosită frecvent în aplicații web cu interfață grafică, soluția implementată nu conține componente de tip View (fișiere .cshtml), deoarece aplicația nu este responsabilă de randarea vizuală a datelor. Prin urmare, controlerele nu returnează pagini HTML, ci obiecte serializate, sub formă de răspunsuri JSON. Pentru controlul accesului, endpoint-urile sunt securizate folosind atributele [Authorize] sau [WithAuthorization(IsAdministrator = true / false)] (un atribut customizat, necesar pentru a verifica tipul de utilizator), astfel încât doar utilizatorii autentificați și autorizați pot accesa anumite resurse. Autentificarea este realizată prin chei de acces (JWT), care au un timp de expirare configurabil, în funcție de nevoile aplicației.

În plus, aplicația a fost dezvoltată pe mai multe microservicii, care, comparativ cu o arhitectură de tip monolit unde toate componentele aplicației se regăsesc într-un singur loc, cea bazată pe microservicii „descompune logica domeniului aplicației, în contexte clar delimitate și consecvente, implementate prin servicii autonome, independente, slab cuplate și care pot fi distribuite separat” (Blinowski, Ojdowska, & Przybyłek, 2022), de asemenea, am avut norocul de a lucra cu ambele arhitecturi, în cadrul job-ului, și diferența se regăsea la nivelul în care se întâmpinau probleme în aplicație: pentru cea monolit, aplicația nu mai era funcțională până la fixare, iar cealaltă, daca un singur microserviciu era afectat, restul funcționau, însa comunicarea cu serviciul afectat și logica aferentă, nu era funcțională până la rezolvarea problemelor.

2.1.2 Baza informațională

O componentă esențială în cadrul oricărui sistem informatic este reprezentată de baza informațională. Aceasta se referă la totalitatea datelor care sunt prelucrate, la modul în care acestea sunt organizate și gestionate, precum și la fluxurile de informații dintre componentele aplicației. În cazul aplicației Certification Hub, scopul principal este de a permite crearea și livrarea cursurilor, evaluarea cunoștințelor prin examene, precum și generarea certificatelor aferente.

Având în vedere complexitatea funcționalităților și diversitatea actorilor implicați – administratori și candidați – s-a impus utilizarea unei soluții hibride de stocare a datelor, bazată pe o combinație de baze de date relaționale și non-relaționale. Din experiența personală acumulată în dezvoltare, dar și din analiza nevoilor sistemului, am optat pentru integrarea următoarelor soluții:

- PostgreSQL este utilizat pentru stocarea datelor structurale, relaționale, precum informațiile despre utilizatori, drepturi de acces, cursuri, examene, răspunsuri la examene și istoricul certificatelor. Baza de date este organizată în tabele distincte pentru fiecare entitate, iar relațiile dintre acestea permit integritatea datelor și interogări complexe.
- MongoDB este folosit pentru stocarea conținutului cursurilor, având în vedere natura dinamică a acestuia. Fiecare curs este reprezentat ca un document JSON ce conține detalii precum titlul, descrierea, statusul și o listă flexibilă de secțiuni structurale (paragrafe, cod, titluri, liste etc.). Această soluție oferă flexibilitate în adăugarea sau modificarea conținutului, fără a fi necesare migrări rigide ale schemelor de date.
- Redis Cache, implementat într-un container Docker separat, este folosit pentru a gestiona temporar cheile de autentificare ale utilizatorilor. La fiecare autentificare, un identificator de sesiune este generat și salvat în Redis, permițând validarea rapidă și eficientă a cererilor ulterioare fără a interoga în mod repetat baza de date relațională. Astfel, se asigură un echilibru între performanță și securitate.

2.2 Proiectarea tehnică

Proiectarea tehnică detaliază aspectele implementării sistemului din punct de vedere fizic și funcțional. Aici sunt descrise structura efectivă a bazei de date, tehnologiile utilizate, procesele de bază care susțin funcționarea platformei și metodele prin care se asigură

eficiența, securitatea și scalabilitatea aplicației. Această etapă transformă modelul logic într-o soluție concretă, pregătită pentru rulare într-un mediu real de utilizare.

2.2.1 Structura fizică a datelor

Structura fizică a datelor reprezintă modul în care informațiile esențiale aplicației sunt organizate, stocate și gestionate în cadrul bazelor de date ale sistemului informatic. În cazul aplicației, arhitectura de date este împărțită în două mari componente: o bază de date relațională PostgreSQL și o bază de date non-relațională MongoDB, completate de o instanță Redis utilizată pentru procesul de autentificare și stocarea temporară a sesiunilor.

Baza de date relațională PostgreSQL este utilizată pentru stocarea datelor cu structură fixă, critice pentru funcționarea platformei: utilizatori, permisiuni, examene, detalii desore cursuri etc. Aceasta este împărțită între cele două API-uri principale: UserManagement API și Management API, fiecare cu propriile tabele și relații definite.

În cadrul UserManagement API, principalele entități sunt:

- User, cu doi discriminatori (AdministratorUser și CandidateUser), permite diferențierea clară între tipurile de utilizatori ;
- Permission, care definește drepturile de acces;
- UserInvitation, utilizat pentru sistemul de înregistrare pe bază de invitație;
- UserPermission, o entitate intermediară ce permite implementarea relației many-to-many între utilizatori și permisiuni.

Astfel, un utilizator poate avea mai multe permisiuni, iar o permisiune poate fi partajată între mai mulți utilizatori. Această relație este implementată eficient printr-o tabelă de legătură. De asemenea, pentru înregistrare, fiecare utilizator poate primi o singură invitație, rezultând o relație one-to-one între User și UserInvitation.

Mai jos voi prezenta entitățile, proprietățile și relațiile acestora cu celelalte entități, sub forma unei diagrame ERD. Se observă că în tabela User, proprietatea „Email” reprezintă un index. Din diagramă reies următoarele relații: User-UserInvitation (un utilizator poate avea o singură invitație în aplicație, o singură invitație poate fi acordată unui utilizator în cadrul aplicației), User-Permission (unul sau mai mulți utilizatori pot avea una sau mai multe permisiuni în cadrul aplicației, una sau mai multe permisiuni pot fi atributie unui singur sau mai multor utilizatori – relație many-to-many, redusă la one-to-many cu ajutorul tabelei UserPermission, astfel un utilizator poate avea mai multe permisiuni-utilizator, iar o permisiune se poate folosi pentru mai multe permisiuni-utilizator).

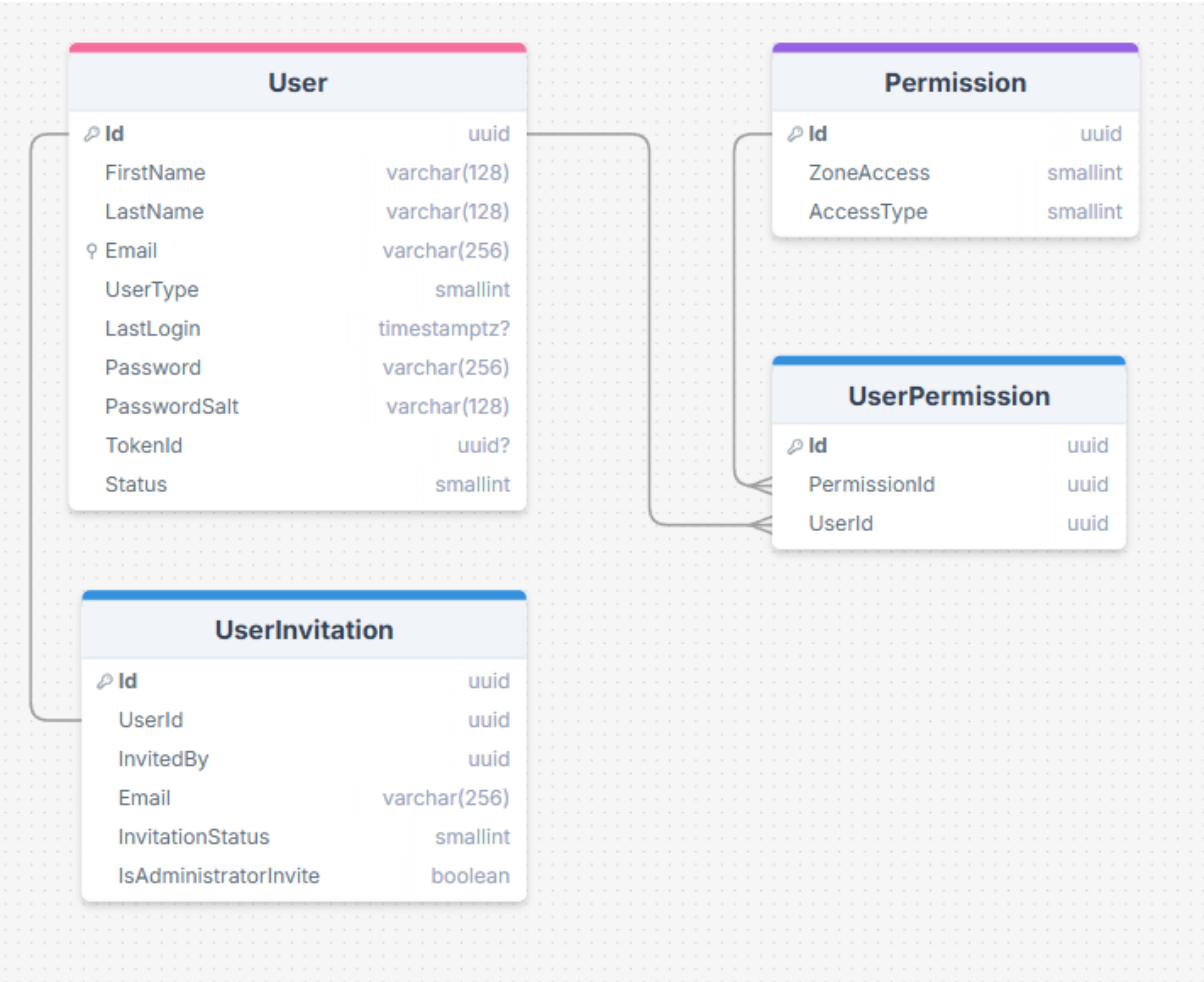


Figura 9. Diagrama ERD (Entity Relationship Diagram) realizată pentru UserManagement API

Diagrama de mai sus s-a realizat folosind un instrument online drawl.sql gasit pe https://drawsql.app/diagrams

Un alt element tehnologic esențial în cadrul acestui API este integrarea Redis, care rulează într-un container Docker dedicat. Redis este utilizat pentru a stoca temporar identificatorii de sesiune (token IDs) generați în urma autentificării. La autentificare, sistemul generează un token JWT, iar identificatorul acestuia este salvat ca și cheie în Redis, facilitând validarea rapidă și sigură a fiecărei cereri ulterioare.

Management API gestionează datele legate de cursuri, examinări și certificări. Principalele entități sunt:

- Course, entitate de bază pentru structurarea cursurilor;
- Tag, folosit pentru clasificarea cursurilor;
- CourseTag, o entitate intermediară ce permite implementarea relației many-to-many între cursuri și etichete;

- CandidateCourseExamination, ce leagă cursurile de candidați și reprezintă înregistrările examinărilor efectuate;
- CandidateCourseExaminationResponse, în care sunt salvate răspunsurile date de candidat într-un examen, legate de examinarea respectivă.

Mai jos voi prezenta entitățile, proprietățile și relațiile acestora cu celelalte entități, sub forma unei diagrame ERD. Din diagramă reies următoarele relații: Course-Tag (un curs poate avea unul sau mai multe etichete, una sau mai multe etichete pot fi aplicate asupra unui singur sau mai multor cursuri – relație many-to-many, redusă la one-to-many cu ajutorul tabelei CourseTag, astfel un curs poate avea una sau multe etichete-curs, iar o etichetă poate fi folosită pentru una sau mai multe etichete-curs), Course-CandidateCourseExamination (un curs va avea o singura examinare, in care se vor specifica numarul de incercari ale candidatului si informatiile necesare unei certificări), CandidateCourseExamination-CandidateCourseExaminationResponse (o examinare a unui curs, poate avea mai multe răspunsuri date de candidat).

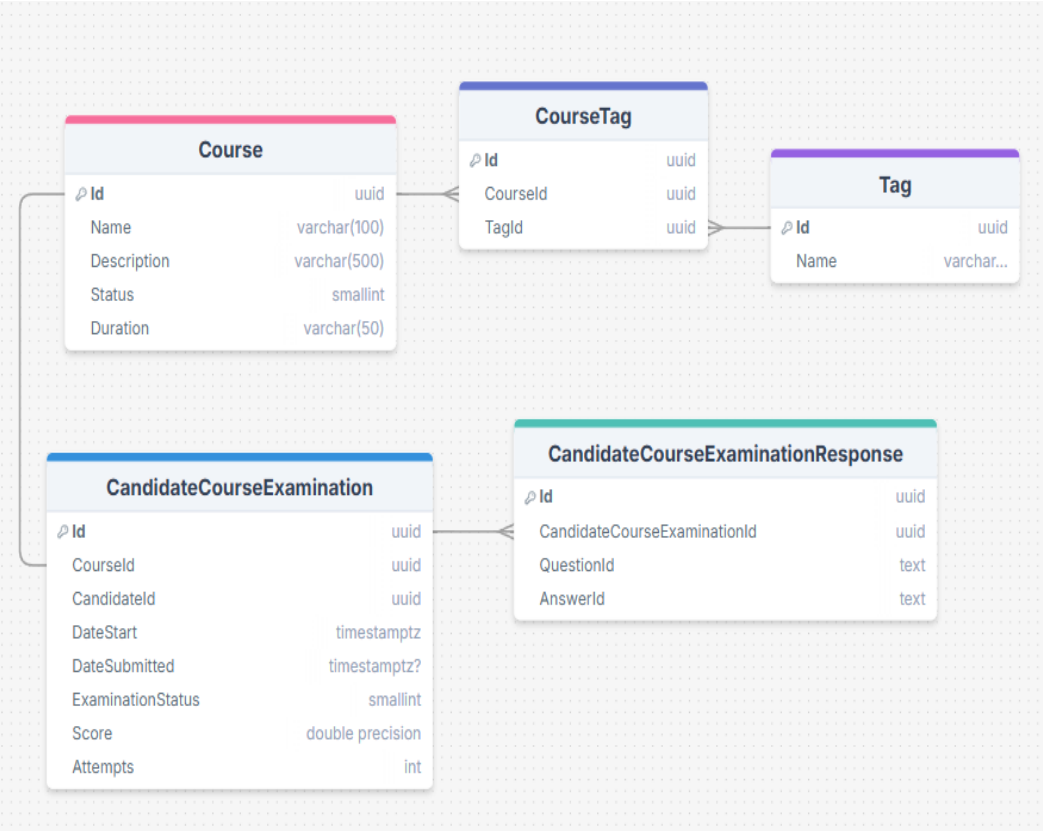


Figura 10. Diagrama ERD (Entity Relationship Diagram) realizată pentru Management API

Diagrama de mai sus s-a realizat folosind un instrument online drawl.sql gasit pe https://drawsql.app/diagrams

Pe lângă această structură relațională, aplicația folosește MongoDB pentru a stoca date dinamice asociate cursurilor, care nu pot fi reprezentate ușor într-un model relațional rigid. În MongoDB, colecția „courses” conține documente care au următoarele câmpuri:

- _id: identificator unic al documentului;
- courseId: referință la cursul asociat din PostgreSQL;
- courseDetails: detalii despre curs (titlu, descriere etc.);
- courseSections: o listă structurată de secțiuni de curs (paragrafe, titluri, cod, liste etc.);
- status: un câmp de tip int care definește dacă versiunea curentă este în stare de „draft” sau „live” (in lucru sau activ).

Această alegere oferă o flexibilitate sporită în manipularea conținutului unui curs, permițând formatorilor să construiască secțiuni în mod liber și să facă modificări fără a compromite structura generală a bazei de date relaționale. Mai jos voi prezenta o poză cu câmpurile unui curs, salvate în MongoDB, și tipul lor de dată:

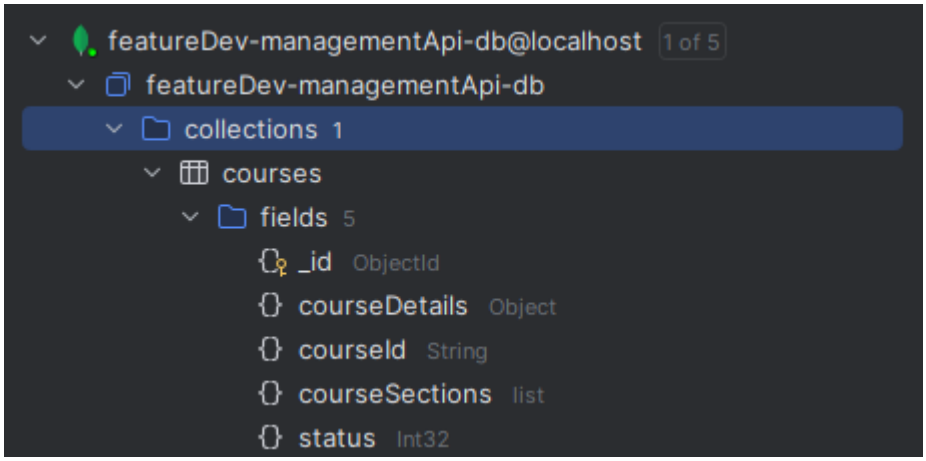


Figura 11. Poză cu structura colecției ce reprezintă cursurile în MongoDB

Iar mai jos voi prezenta un exemplu de conținut a unui curs, și anume câmpul courseSections: (Conținutul în MongoDB a unui curs, este destul de mare, am să afișez doar o imagine simplă, cu o parte din acesta, pentru a evita umplerea de pagini cu poze)

```
[
  {
    "courseSection": {
      "_id": "1742661374",
      "title": "About threads",
      "items": [
        {
          "item": {
            "_id": "1742661404",
            "type": "TitleAndText",
            "title": "What is a thread?",
            "text": "A thread represents the smallest unit of execution within a process, or it represents a set of instructions that the CPU executes.\nThe
             operating system (OS) decides when and how much CPU time a thread receives and also schedules execution of a process at a thread level."
          }
        },
        {
          "item": {
            "_id": "1742661575",
            "type": "TitleAndText",
            "title": "What is a process?",
            "text": "A process represents an instance of a running program, e.g. Notepad."
          }
        }
      ]
    }
  },
  {
    "courseSection": {
      "_id": "1742661651",
      "title": "Threads and .NET",
      "items": [
        {
          "item": {
            "_id": "1742661663",
            "type": "TitleAndText",
            "title": "How does a Thread work in .NET framework?",
            "text": ".NET runtime provides additional management, abstraction and APIs to work with threads in a more convenient way.\n"
          }
        }
```


Figura 12. Poză cu conținut de curs din MongoDB

2.2.2 Procese

În cadrul acestei subsecțiuni, vor fi prezentate patru dintre cele mai importante procese care susțin funcționalitatea aplicației Certification Hub. Aceste procese reflectă logica din spatele interacțiunii utilizatorilor cu platforma și asigură atât controlul asupra accesului, cât și o experiență fluidă și coerentă pentru fiecare categorie de utilizator.

Unul dintre cele mai importante procese este cel de înregistrare pe bază de invitație. Această abordare a fost aleasă în ciuda unei înregistrări publice clasice, tocmai pentru a menține un control strict asupra utilizatorilor care accesează platforma, permițând doar celor autorizați să se înregistreze. Astfel, se evită crearea de conturi false, spam sau accesul persoanelor care nu aparțin grupurilor țintă.

Procesul debutează cu un administrator care accesează secțiunea de gestionare a invitațiilor. Aici, poate genera o invitație destinată fie unui candidat, fie unui alt administrator, prin introducerea unei adrese de email valide. Sistemul trimite automat un email personalizat, care conține un link unic către formularul de înregistrare.

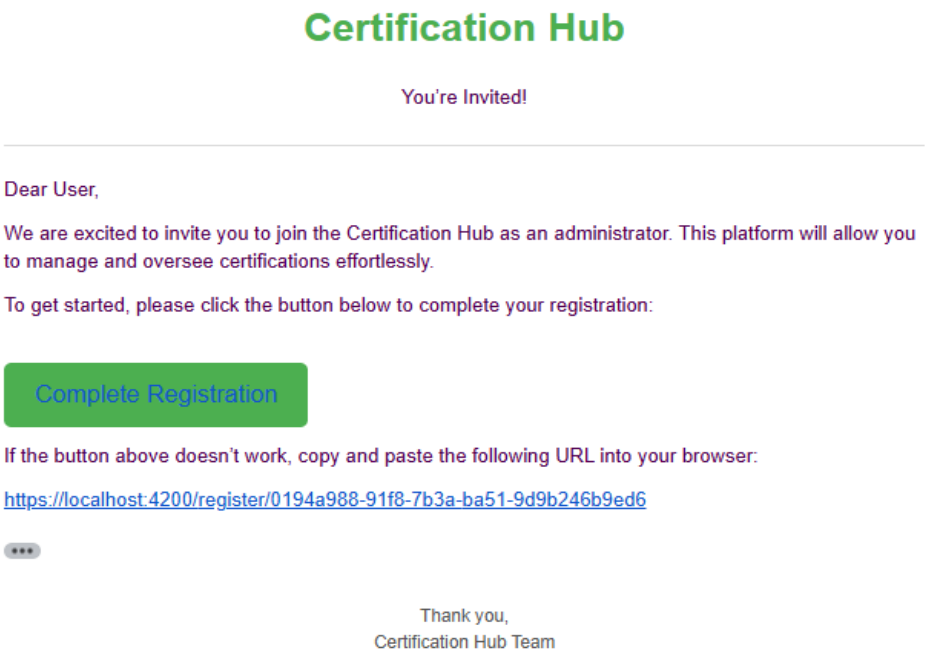


Figura 13. Prezentarea unui mail de înregistreare pe platforma Certification Hub

În figura de mai sus, se observă un mail cu o invitație destinată unui viitor administrator în cadrul platformei Certification Hub.

Când utilizatorul accesează link-ul, este redirecționat către pagina de înregistrare, unde i se vor precompleta anumite date (precum email-ul) și i se va solicita completarea datelor personale (nume, prenume, parolă). În fundal, aplicația validează existența invitației și tipul acesteia, după care creează un utilizator nou, asociat cu un set de permisiuni implicite: permisiuni complete în cazul administratorilor, sau permisiuni limitate în cazul candidaților. După înregistrare, utilizatorul este redirecționat automat către pagina de login.

Această metodologie nu doar că adaugă un strat suplimentar de securitate și control, dar contribuie și la menținerea unei structuri organizate în baza de date, evitând înregistrările redundante sau neautorizate.

În urma înregistrării, utilizatorul este redirecționat în zona de autentificare, unde, în urma completarii formularului cu credențialele necesare (email și parolă), acesta este redirecționat la pagina principală a aplicației (dashboard). În spate, pentru sesiunea utilizatorului respectiv, se salveaza un id al cheii de access, generate în urma autentificării și se salvează într-o instanță de Redis, creată cu Docker (despre care voi discuta în umătoarele capitole).

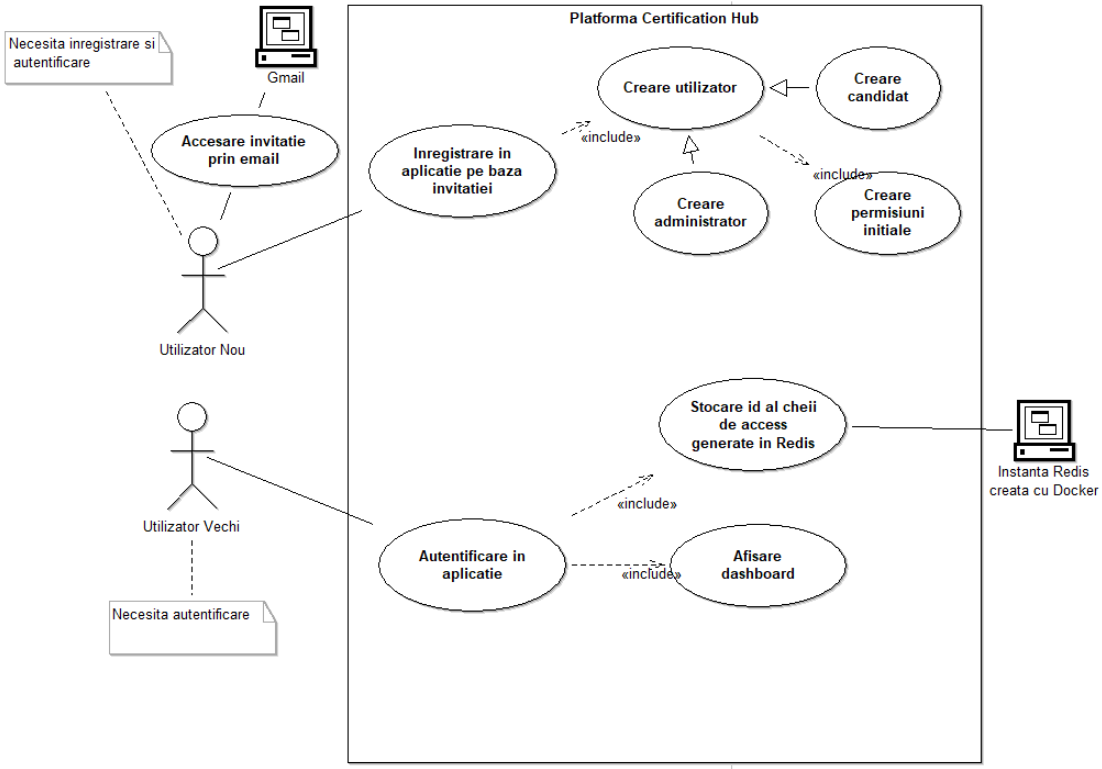


Figura 14. Diagrama use-case ce evidențiază procesul de înregistrare și autentificare

Diagrama de mai sus s-a realizat în aplicația Bee-Up 1.7 folosită și în cadrul seminariilor de Analiza și proiectarea sistemelor informatice.

Un al doilea proces esențial este gestionarea cursurilor, realizată exclusiv de către utilizatorii de tip administrator. Spre deosebire de o abordare clasică în care modificările unui curs activ pot fi făcute direct, în cadrul ,,Certification Hub” se implementează un mecanism de versionare.

Inițial, un curs nou este salvat în baza de date MongoDB (bază de date non relațională) cu un status "draft" (în lucru), iar detaliile principale (denumire, descriere, etichete, durată) sunt salvate în PostgreSQL (bază de date relațională).

Administratorul poate adăuga secțiuni cu paragrafe, cod, liste sau informații legate de examinare, toate fiind gestionate dinamic și salvate incremental. Când cursul este considerat pregătit, administratorul poate aplica modificările, moment în care cursul este salvat cu statusul "active".

De menționat faptul că utilizatorul administrator poate accesa un curs activ și să îl transforme ulterior într-o variantă de lucru, prin accesarea butonului de ,,Create Draft”. Orice actualizare de date se va reflecta in documentul aflat în lucru, doar în MongoDB, apoi, în momentul în care se accesează butonul de ,,Save Changes”, detaliile principale (actualizate sau nu) se vor reflecta și în PostgreSQL. De asemenea prin accesarea butonului ,,Discard Changes”, varianta în lucru a cursului se va șterge din MongoDB, și nicio schimbare nu va apărea la nivel de PostgreSQL.

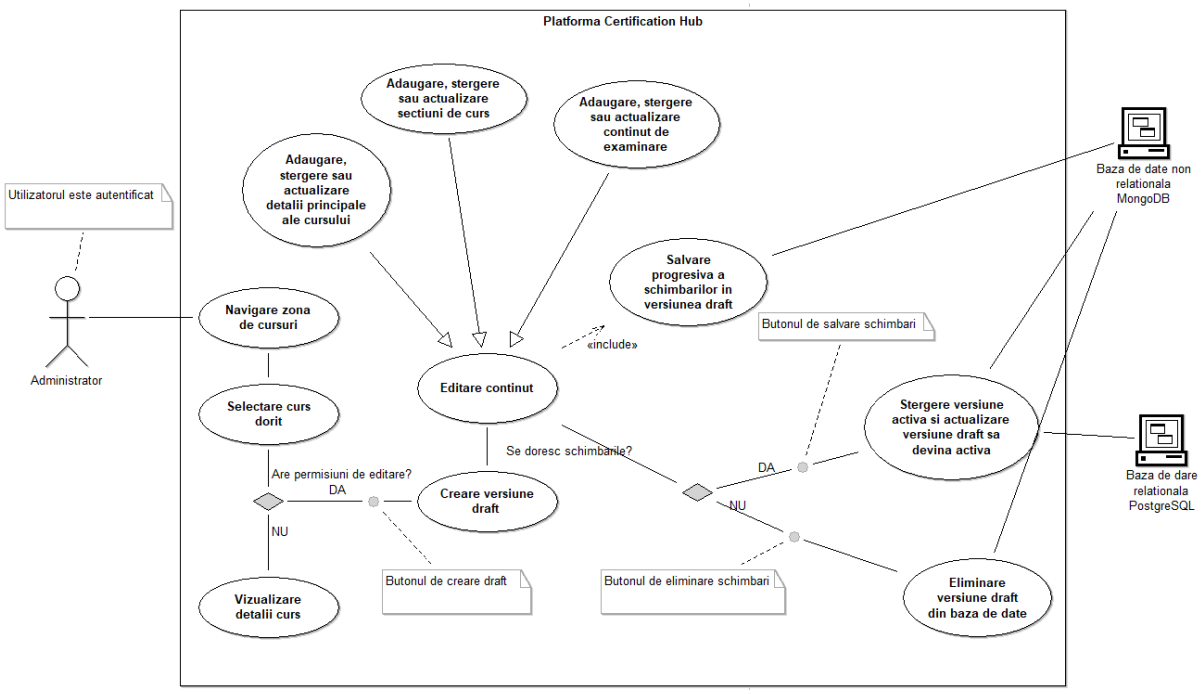


Figura 15. Diagrama use-case ce evidențiază procesul de modificare a cursurilor

Diagrama de mai sus s-a realizat în aplicația Bee-Up 1.7 folosită și în cadrul seminariilor de Analiza și proiectarea sistemelor informatice.

Al treilea proces urmărit este cel de evaluare și generare a certificatelor, adresat exclusiv candidaților. După ce un candidat accesează un curs, are posibilitatea de a parcurge conținutul acestuia în mod secvențial. La final, dacă cursul conține un examen activ, candidatul poate accesa secțiunea de testare.

Examenul este compus dintr-un număr variabil de întrebări cu variante multiple de răspuns, unele corecte și altele greșite. Candidatul trebuie să selecteze răspunsurile într-un timp limitat, stabilit de către administrator. După finalizare, un sistem de feedback automat îi afișează:

- punctajul total obținut;
- fiecare întrebare cu răspunsul marcat corect sau greșit (cu evidențiere vizuală – roșu/verde);
- scorul per întrebare.

Dacă examenul este promovat, candidatul va avea acces în secțiunea "Certificări", unde sunt listate toate examenele promovate. La un simplu click, utilizatorul poate descărca certificatul în format PDF, validat cu datele cursului și ale candidatului. Acest proces este conceput pentru a fi intuitiv, motivant și transparent.

Un alt mecanism esențial în funcționarea aplicației este cel legat de validarea cererilor HTTP, în funcție de permisiunile utilizatorilor. Pentru a proteja resursele aplicației, toate endpoint-urile Backend sunt securizate prin atribute Authorize, unele fiind personalizate. Fiecare cerere verifică:

- dacă utilizatorul este autentificat (JWT valid);
- dacă utilizatorul are dreptul să acceseze ruta respectivă;
- dacă permisiunile atașate utilizatorului includ acțiunea solicitată (ex: editare curs, vizualizare invitații etc.).

Acest mecanism de control al accesului nu este prezent doar la nivel de Backend, ci și în cadrul Frontend-ului, unde a fost implementat un sistem de protecție al rutelor folosind așa-numitele ,,Route Guards” (funcționalități specifice Angular). Acestea asigură faptul că utilizatorii accesează doar secțiunile aplicației pentru care sunt autorizați.
Tipurile implementate sunt:

- AuthGuard – Acest gardian protejează rutele destinate doar utilizatorilor autentificați. De exemplu, dashboard-ul, profilul, gestionarea utilizatorilor, cursurilor și certificărilor pot fi accesate doar după autentificare validă. Dacă utilizatorul nu este autentificat, acesta este redirecționat automat către pagina de login (/login).
- AnonymousGuard – Acest tip de gardian are rolul opus: blochează accesul utilizatorilor autentificați către paginile de login sau

înregistrare. Astfel, se evită situațiile în care un utilizator deja logat încearcă să acceseze accidental o zonă pentru utilizatori neautentificați. Acești gardieni sunt integrați direct în configurația rutelor aplicației Angular, iar protecția este aplicată cu ajutorul proprietății ,,canActivate”, care determină dacă accesul este permis sau nu. De exemplu: rutele dashboard, profile, users, courses, candidate-course-examination/:id și certifications sunt protejate prin AuthGuard, asigurând accesul doar pentru utilizatori logați, iar login și register sunt protejate cu AnonymousGuard, fiind accesibile doar utilizatorilor care nu sunt autentificați.

2.3 Tehnologii specifice

2.3.1 Limbajul C#

Aplicația a fost construită folosind pentru Backend limbajul de programare C#, acesta reprezintă ,,un limbaj de programare simplu, modern, orientat pe obiecte și sigur din punct de vedere al tipurilor, care îmbină productivitatea ridicată specifică limbajelor de dezvoltare rapidă a aplicațiilor cu puterea brută a limbajelor C și C++.” (Anders , Scott , & Peter , 2003).

2.3.2 ASP.NET Core

ASP.NET Core este un framework cross-platform, ceea ce înseamnă că aplicațiile dezvoltate cu ajutorul său pot rula pe mai multe sisteme de operare, precum Windows, Linux și macOS, fără modificări semnificative în cod. Este un framework high-performance, optimizat pentru viteze ridicate de execuție și scalabilitate, ceea ce îl face potrivit pentru aplicații care trebuie să gestioneze un număr mare de utilizatori sau operațiuni simultane. ASP.NET Core permite dezvoltatorilor să creeze aplicații web**,** servicii și API-uri REST eficiente, care să comunice între aplicații sau componente externe (Daniel , Rick , & Shaun , 2025).

2.3.3 Serviciul REST

Serviciul REST (Representational State Tranfser, tradus Transfer de stare reprezentațională), folosit în cadrul aplicației dezvoltate, reprezintă ,,o arhitectură utilizată pentru proiectarea serviciilor care pot fi consumate pe diverse platforme și în medii diferite, având ca scop asigurarea interoperabilității și susținerea funcționării eficiente a World Wide Web (WWW)”. API-urile de tip REST sunt ,,frecvent utilizate în proiectarea arhitecturilor de tip microservicii" și se mai numesc ,,web APIs”, acestea sunt formate din endpoint-uri, care reprezintă ,,o funcționalitate concretă asociată unui process business”, accesibile prin

intermediul protocolului HTTP, ce utilizeaza verbe standardizate GET, POST, PUT și DELTE, iar invocarea unui endpoint „se realizează printr-un identificator de resursă unic, numit URI sau Uniform Resource Identifier” (Adeel , Mohammed, Cagatay , & Deepti , 2022)

2.3.4 Entity Framework (EF) Core

În cadrul aplicației, pentru a lucra corespunzător cu o baze de date, a mapa modele existente (de exemplu: User, Permission, Course, Tag etc.) și a defini relațiile dintre ele, într-un mod elegant și ușor de configurat, am folosit Entity Framework (EF) Core. Acest framework reprezintă „un ORM (Object-Relational Mapping, tradus Mapare Obiect-Relațională), open-source, care simplifică interacțiunea dintre codul aplicației și bazele de date relaționale’’ (Jayaraman & Siddharth, 2025). Am ales acest ORM, deoarece, implementează un sistem de schimbări aplicate la nivel de modele (identificând daca s-a actualizat, adăugat sau șters o entitate), realizând „automat modificările efectuate asupra entităților, generând comenzile SQL corespunzătoare pentru actualizarea bazei de date, astfel să reflecte starea aplicației” (Jayaraman & Siddharth, 2025) și datorită optimizărilor interogărilor realizate către bazele de date. Totodată acesta pune la dizpoziție „un sistem de migrații automate, care permite dezvoltatorilor să gestioneze modificările de schemă ale bazei de date în timp, acest lucru facilitând adaptarea bazei de date la noice cerințe fără pierderi de date. (Jayaraman & Siddharth, 2025)

2.3.5 Docker

Mai sus am menționt utilizarea Docker în cadrul aplicației mele, prin crearea unui container dedicat unei instanțe de Redis utilizată în cadrul procesului de autentificare. Docker reprezintă o platforma open-source, folosită pentru dezvoltarea, livrarea și rularea aplicațiilor, facilitând separarea aplicațiilor de infrastructură. Această platformă ne permite să rulăm o aplicație într-un cadru izolat, numit container, creat pe baza unei imagini ce reprezintă o schiță cu instrucțiuni necesare pentru rularea acestuia. (Docker Inc)

2.3.6 Redis

În cadrul aplicației, am utilizat Redis pentru gestionarea sesiunilor de autentificare, mai exact pentru a stoca identificatorii token-urilor de acces (ca și chei) și token-urile propriu-zise (ca valori). Acest mecanism permite validarea rapidă a sesiunilor active, precum și invalidarea lor atunci când este necesar — un aspect esențial pentru menținerea securității aplicației și a

controlului asupra accesului. Redis este o bază de date NoSQL, orientată pe stocarea în memorie, proiectată pentru a oferi performanțe ridicate. Este folosită frecvent pentru operațiuni de caching, căutari vectoriale și ca bază de date. (Redis Ltd)

2.3.7 Angular

Din punct de vedere al interfeței vizuale ale aplicației, Frontend-ul a fost realizat cu ajutorul Angular. Acesta reprezintă ,,un framework modern utilizat pentru dezvoltarea aplicațiilor web avansate, pe partea de client" și are la bază ,,limbajul TypeScript, care oferă o structurare mai clară a codului și verificarea tipurilor în timpul compilării". Angular a fost dezvoltat de Goolge, iar codul într-o aplicație de genul este ,,organizat ăe componente, fiecare cu propriul fișier HTML, CSS / SCSS și logică scrisă în TypeScript", prin acest mod se promovează ,,o separare clară a responsabilităților și permite o dezvoltare modulară". De asemenea este foarte flexibil și se îmbină foarte bine cu ASP.NET Core ,,combinarea cu ASP.NET Core s-a evidențiat prin performanță și organizare clară" (Cincovic, Delcev, & Draskovic, 2019).

2.3.8 SCSS

Pe partea de Frontend, am menționat faptul că am folosit SCSS (nu CSS). SCSS extinde CSS-ul și adaugă funcționaliăți extra, precum: variable, bucăți de cod reutilizabile (numite mixins) și o structură ierarhică, asemănătoare cu cea de HTML (Sass).

2.3.8 PostgreSQL

În cadrul proiectului am utilizat o bază de date relațională, pentru a stoca informațiile de bază, non-dinamice, precum: denumiri, descrieri, nume, date utilizatori, folosind PostgreSQL. PostgreSQL este „un sistem de gestiune a bazelor de date relaționale open-source, care pune accent pe extensibilitate […] compatibil cu sisteme de operare moderne precum Windows […] oferă funcționalități de replicare, moșternirea tabelelor, precum și suport pentru extensii precum pgAdmin" (Juba, Vannahme, & Volkov, 2015). De asemenea, din experiența acumulată la locul de muncă, am observat că PostgreSQL permite definirea de tabele externe prin intermediul mecanismului FDW (Foreign Data Wrappers). Această funcționalitate este utilă în arhitecturile bazate pe microservicii, permițând unui serviciu să acceseze date aflate în baze de date asociate altor microservicii, fără a fi nevoie de duplicarea informației.

2.3.8 MongoDB

În cadrul aplicației, pe lângă baza de date relațională utilizată pentru gestionarea datelor structurale, am implementat și o bază de date non-relațională MongoDB, pentru a stoca

informațiile dinamice asociate cursurilor și examinării. Concret, datele privind structura secțiunilor de curs, conținutul acestora și examinarea aferentă sunt persistate sub formă de documente în format JSON. Această decizie a fost motivată de nevoia de flexibilitate, deoarece elementele care compun cursurile (titluri, descrieri, liste, cod, întrebări și răspunsuri) sunt variabile de la un curs la altul și ar fi dificil de modelat eficient într-o schemă relațională. Un avantaj al bazelor de date non relaționale este faptul că ,,acestea sunt mai scalabile și oferă o performanță superioară, adresând probleme pe care modelul relațional nu le poate gestiona eficient, cum ar fi volume mari de date structurate, semi-structurate sau nestrucurate, iterații rapide și dezvoltare agilă" (Győrödi, Sotoc, & Győrödi, 2015).

3. Implementarea sistemului informatic

În acest capitol este detaliat modul în care a fost realizată implementarea practică a aplicației Certification Hub. Se pornește de la definirea modelelor și crearea tabelelor prin intermediul Entity Framework Core, fiind utilizată o bază de date relațională pentru păstrarea datelor esențiale. Se evidențiază importanța programării asincrone în asigurarea performanței, precum și integrarea containerului Redis pentru stocarea și validarea sesiunilor de autentificare. În continuare sunt descrise funcționalitățile esențiale ale aplicației, alături de cele remarcabile care o diferențiază de alte soluții similare. Este explicat modul în care are loc comunicarea cu serviciile web prin intermediul API-urilor și sunt prezentate instrumentele și tehnologiile folosite pe tot parcursul procesului de dezvoltare.

3.1 Crearea tabelelor utilizând EF Core

Fiecare microserviciu dispune de propriul context (DbContext), care reflectă structura bazei de date asociate. Acest context este definit în cadrul unei clase specifice, în care sunt configurate entitățile, relațiile dintre ele, constrângerile, dar și eventualele discriminatori sau indexări:

```
public class DataContext(DbContextOptions options, ILoggerFactory loggerFactory) :
DbContextBase(options)
{
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    SetIndexes(modelBuilder);
    SetDiscriminators(modelBuilder);
    SetModelsRelations(modelBuilder);
}

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{

}

private static void SetIndexes(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>()
        .HasIndex(user => user.Email)
        .IsUnique();
}

private static void SetDiscriminators(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>()
        .HasDiscriminator(u => u.UserType)
        .HasValue<CandidateUser>(UserType.Candidate)
        .HasValue<AdministratorUser>(UserType.Administrator)
        .HasValue<User>(UserType.Unknown);
}

private static void SetModelsRelations(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>()
        .HasMany(user => user.UserPermissions)
        .WithOne(permission => permission.User)
        .HasForeignKey(permission => permission.UserId)
```

```
            .OnDelete(DeleteBehavior.Cascade);

        modelBuilder.Entity<Permission>()
            .HasMany(permission => permission.UserPermissions)
            .WithOne(user => user.Permission)
            .HasForeignKey(permission => permission.PermissionId)
            .OnDelete(DeleteBehavior.Cascade);

        modelBuilder.Entity<User>()
            .HasOne(user => user.UserInvitation)
            .WithOne(invitation => invitation.User)
            .HasForeignKey<UserInvitation>(invitation => invitation.UserId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}
```

Mai sus am prezentat codul sursă din cadrul UserManagement.Api. Codul dispune metode care setează relațiile corespunzătoare dintre entități, care setează discriminatori asupra tabeleui ,,User” și anume ,,Candidate” și ,,Administrator” (pentru a nu crea două tabele separate, una pentru fiecare tip, ci a avea una singură cu o configurație care permite diferențierea tipului de utilizatori), și o metodă pentru setarea unui index asupra email-ului utilizatorului (pentru căutarea mai rapidă).

,,DbContextBase” reprezintă o clasă de bază definită de mine, care moștenește din clasa DataContext, oferită de EF Core, și adaugă niște extensii precum: preluarea numelui bazei de date și a tabelelor utilizate.

Fiecare entitate care va fi mapată la un tabel in baza de date, moștenește o interfață ,,IEntity”, cu ajutorul acestei interfețe am reușit sa diferențiez modelele folosite la nivel de bază de date, față de restul, un exemplu de clasă este:

```
public class User : IEntity
{
    public Guid Id { get; set; }
    [Column(TypeName = "varchar(128)")]
    public string FirstName { get; set; } = string.Empty;
```

```
    [Column(TypeName = "varchar(128)")]
    public string LastName { get; set; } = string.Empty;
    [Column(TypeName = "varchar(256)")]
    public string Email { get; set; } = string.Empty;
    [Column(TypeName = "varchar(256)")]
    public string Password { get; set; } = string.Empty;
    public UserStatus Status { get; set; } = UserStatus.Unknown;
    public UserType UserType { get; set; }
    public DateTime? LastLogin { get; set; }
    public List<UserPermission>? UserPermissions { get; set; } = [];
    public UserInvitation? UserInvitation { get; set; }
    public Guid? TokenId { get; set; }
    [Column(TypeName = "varchar(128)")]
    public string PasswordSalt { get; set; } = string.Empty;
}
```

De asemenea se observă anumite anotări folosite din cadrul EF Core, pentru a determina lungimea proprietăților de tip „string”.

Pentru a comunica cu baza de date, și a realiza interogări, am creat o clasă generică numită „GenericRepository”, care implementează o interfață „IGenericRepository”, și funcționează pe elementele de tip „IEntity”. Această clasă definește operații CRUD și funcționalități de sortare, paginare și filtrare. (nu voi intra în prea multe detalii în zona aceasta, deoarece este foarte comună).

Totodată am menționat folosirea unei clase „UnitOfWork” în cadrul fiecărui microserviciu, care moștenește „IUnitOfWork” și definește clase concrete de tip repository pentru fiecare entitate. Cu ajutorul acestei clase, am un loc din care pot accesa orice tip de repository, iar toate metodele acestuia, se vor folosi de entitatea pentru care a fost implementată.

```
public interface IUnitOfWork
{
    public IGenericRepository<AdministratorUser> AdministratorUserRepository { get; }
    public IGenericRepository<CandidateUser> CandidateUserRepository { get; }
    public IGenericRepository<UserInvitation> UserInvitationRepository { get; }
```

```
    public IGenericRepository<User> UserRepository { get; }
    public IGenericRepository<UserPermission> UserPermissionRepository { get; }
    public IGenericRepository<Permission> PermissionRepository { get; }
    public Task CompleteAsync();
    public IRepositoryContext BeginNewContext();
}
```

Un exemplu în care am folosit această clasă îl reprezintă o simplă interogare de tip „SELECT” asupra tabelei „User”:

```
var candidates =
_unitOfWork.CandidateUserRepository.SearchWithCountAndFilterAsync(candidateUserFilt
er, searchSettings, selector);
```

(Caută candidații, îi filtrează, aplică paginare și selectează proprietățile cerute).

3.2 Programarea asincronă

În cadrul dezvoltării aplicației, am utilizat programarea asincronă pentru a îmbunătăți performanța și fluiditatea interacțiunii cu baza de date, fără a bloca firul de execuție principal al aplicației. Deși nu a fost implementat un mecanism complex sau avansat de concurență, am folosit principiile oferite de limbajul C# prin cuvintele cheie „async” și „await”. Majoritatea funcționalităților aplicației sunt implementate asincron. Am folosit de asemenea biblioteca de LINQ (Language Integrated Query) asincronă, cu metode precum: „ToListAsync”, „SingleOrDefaultAsync” sau „AsAsyncEnumerable” pentru a reduce timpul de așteptare în cazul interacțiunilor cu baza de date, exemplu:

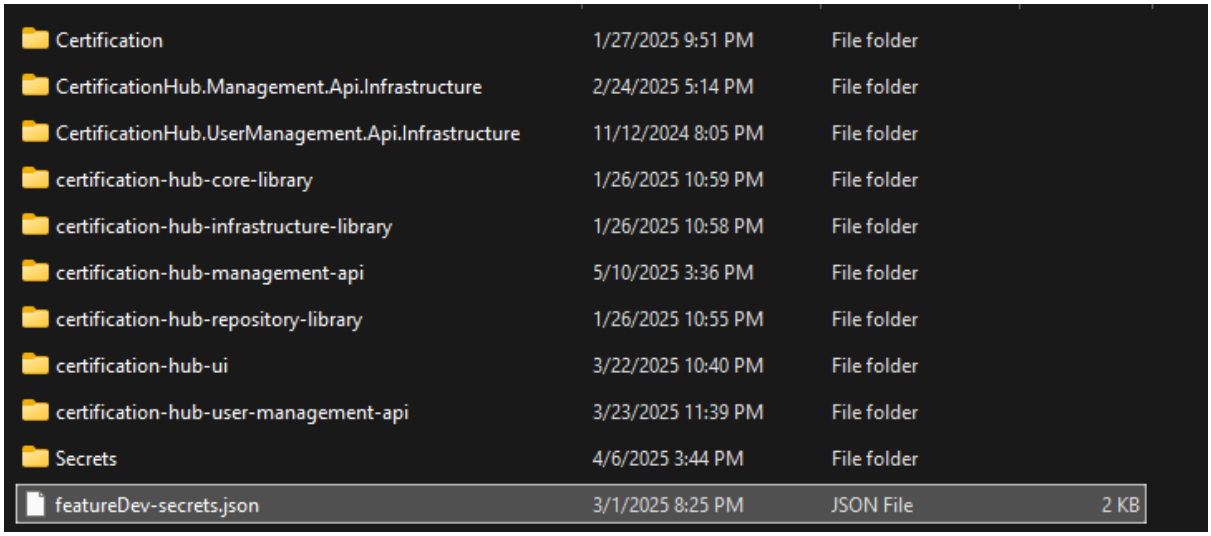
```
var permissions = await _permissionService
    .SearchPermissionsAsync(new PermissionFilter
    {
        ReturnUnknown = false
    }, new SearchSettings(), p => p)
    .Entities
    .ToListAsync();
```

3.3 Gestiunea secretelor

Aplicația a fost gândită într-un mod prin care, mă pot folosi de diferite secrete (conexiuni la nivel de baze de date, chei de criptare pentru JWT, configurări de email sau Redis), în funcție de mediul de lucru folosit. Acest mediu de lucru îl definesc in zona ,,launchSettings.json" al aplicației, prin schimbarea valorii variabilei ,,ASPNETCORE_ENVIRONMENT", folosind o convenție: mediuDeLucru + Local (featureDevLocal, devLocal, prodLocal etc.).

```
"CertificationHub.UserManagement.Api.Presentation" : {
  "commandName": "Project",
  "launchBrowser": true,
  "commandLineArgs": "app-listening-port=10010 gRPC-listening-port=11010",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT" : "featureDevLocal"
  },
  "applicationUrl": "http://localhost:10010/userManagementApi/swagger",
  "dotnetRunMessages" : true
}
```

Aceste secrete sunt notate într-un fișier cu extensia json, cu un nume care respectă convenția de mai sus, plasat în radăcina proiectului:



Certification	1/27/2025 9:51 PM	File folder	
CertificationHub.Management.Api.Infrastructure	2/24/2025 5:14 PM	File folder	
CertificationHub.UserManagement.Api.Infrastructure	11/12/2024 8:05 PM	File folder	
certification-hub-core-library	1/26/2025 10:59 PM	File folder	
certification-hub-infrastructure-library	1/26/2025 10:58 PM	File folder	
certification-hub-management-api	5/10/2025 3:36 PM	File folder	
certification-hub-repository-library	1/26/2025 10:55 PM	File folder	
certification-hub-ui	3/22/2025 10:40 PM	File folder	
certification-hub-user-management-api	3/23/2025 11:39 PM	File folder	
Secrets	4/6/2025 3:44 PM	File folder	
featureDev-secrets.json	3/1/2025 8:25 PM	JSON File	2 KB

Figura 16. Prezentarea fișierului de secrete în cadrul aplicației

În librăria ,,core" am definit un serviciu numit ,,SecretsService", care se folosește de o clasă numită ,,AppSecrets", unde sunt grupate toate entitățile folosite pentru configurare. Această clasă realizează urmatoarele:

- Încarcă fișierul de secrete, care conține sufixul de ,,secrets.json";
- Deserializează conținutul în obiectul ,,AppSecrets";
- Oferă acces centralizat la proprietățile importante din acea configurație.

```
using CertificationHub.Core.Library.Entities;
using CertificationHub.Core.Library.Extensions;
using CertificationHub.Core.Library.Models;
using CertificationHub.Core.Library.Security.Jwt;
using Microsoft.Extensions.Hosting;
using Newtonsoft.Json;

namespace CertificationHub.Core.Library.Secrets;

public class SecretService(IHostEnvironment hostEnvironment) : ISecretService
{
    private const string LocalSecretsFileNameSuffix = "secrets.json";
    private AppSecrets? _appSecrets;

    public async Task RefreshSecretsAsync()
    {
        try
        {
            _appSecrets = await GetSecretsAsync();
        }
        catch (Exception exception)
        {
            Console.WriteLine($"{exception.Message}\r\b{exception.StackTrace}");
        }
    }
    async Task ISecretService.InitAsync()
```

```
    {
        _appSecrets = await GetSecretsAsync();
    }

    public MongoConnection? MongoConnection => _appSecrets?.MongoConnection;
    public string? PostgresSqlUserManagementApiConnectionString =>
_appSecrets?.PostgresSqlUserManagementApiConnectionString;
    public string? PostgresSqlManagementApiConnectionString =>
_appSecrets?.PostgresSqlManagementApiConnectionString;
    public JwtSettings? JwtSettings => _appSecrets?.JwtSettings;
    public RedisConnection? RedisConnection => _appSecrets?.RedisConnection;
    public EmailConfiguration? EmailConfiguration => _appSecrets?.EmailConfiguration;

    private async Task<AppSecrets> GetSecretsAsync()
    {
        var serializedSecrets = string.Empty;

        if (hostEnvironment.IsLocalDevelopmentEnvironment())
        {
            var localEnvironment =
hostEnvironment.EnvironmentName.Remove(hostEnvironment.EnvironmentName.LastInde
xOf("Local", StringComparison.OrdinalIgnoreCase));
            var secretFile = FindSecretFile(hostEnvironment.ContentRootPath,
localEnvironment);
            serializedSecrets = await File.ReadAllTextAsync(secretFile);
        }

        var secrets = JsonConvert.DeserializeObject<AppSecrets>(serializedSecrets);

        if (secrets is null)
        {
            throw new Exception("Could not retrieve secrets from current file");
        }
```

```
        return secrets;
    }

    private string FindSecretFile(string? startPath, string localEnvironment)
    {
        var fileName = $"{localEnvironment}-{LocalSecretsFileNameSuffix}";

        while (startPath is not null)
        {
            var secretsFilePath = Path.Combine(startPath, fileName);
            if (File.Exists(secretsFilePath))
            {
                return secretsFilePath;
            }

            startPath = Directory.GetParent(startPath)?.FullName;
        }

        throw new Exception($"Specified file {fileName} could not be found");
    }
}
```

Această clasă de secrete este folosită în cadrul librăriilor, inițializate la start, de exemplu librăria de securitate:

```
public class SecurityLibrary : ILibraryInitialisationServiceWithProvider
{
    public void RegisterLibraryServicesInContainer(IServiceCollection services,
Func<IWebHost> provider)
    {
        services.AddSingleton<IJwtGenerator, JwtGenerator>();
        AddJwtTokenServices(services, provider);
    }

    public Task InitLibraryAsync(IServiceProvider serviceProvider)
```

```
    {
      return Task.CompletedTask;
    }

    private static void AddJwtTokenServices(IServiceCollection serviceCollection,
Func<IWebHost> getHost)
    {
       serviceCollection.AddAuthentication(options =>
          {
             options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
             options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
          })
          .AddJwtBearer(options =>
          {
             var serviceProvider = getHost().Services;
             var secretService = serviceProvider.GetRequiredService<ISecretService>();
             var jwtSettings = secretService.JwtSettings;
             var secretKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings!.SecretKey));

             options.RequireHttpsMetadata = jwtSettings.RequireHttpsMetadata;
             options.SaveToken = true;
             options.TokenValidationParameters = new TokenValidationParameters
             {
                ClockSkew = TimeSpan.Zero,
                IssuerSigningKey = secretKey,
                ValidateIssuerSigningKey = jwtSettings.ValidateIssuerSigningKey,
                ValidateLifetime = jwtSettings.ValidateLifetime,
                RequireExpirationTime = jwtSettings.RequireExpirationTime,
                TokenDecryptionKey = new
SymmetricSecurityKey(Encoding.Default.GetBytes(jwtSettings.SecretKey)),
                ValidateAudience = true,
                ValidateIssuer = true,
                ValidAudience = "Certification Hub",
```

```
                ValidIssuer = "Certification Hub"
            };
        });
    }
}
```

Clasa de secrete îmi permite să accesez informațiile necesare pentru a configura conexiunea la baza de date, setarea securității și așa mai departe.

Un alt exemplu ar fi modul în care setez conexiunea la baza de date, în intermediul claselor de configurare ale aplicației:

```
internal static void AddDataContext(this IServiceCollection serviceCollection)
{
    const string MigrationAssembly = "CertificationHub.UserManagement.Api.Infrastructure";

    serviceCollection.AddScoped(serviceProvider =>
    {
        var secretsService = serviceProvider.GetRequiredService<ISecretService>();
        var connectionString =
secretsService.PostgresSqlUserManagementApiConnectionString;
        var optionsBuilder = new DbContextOptionsBuilder();

        optionsBuilder.UseNpgsql(connectionString, options =>
        {
            options.MigrationsAssembly(MigrationAssembly);
        });

        return new DataContext(optionsBuilder.Options,
serviceProvider.GetRequiredService<ILoggerFactory>());
    });
}
```

3.4 Gestionarea Redis

Așa cum am menționat în capitolul în care am prezentat arhitectura proiectului, am folosit o instanță de Redis creată cu ajutorul containerelor definite de Docker. Aceasta a fost creată prin definirea unei imagini, cu ajutorul unei comenzi simple:

docker run --name redis-db -p 6379:6379 -d redis

Așa am creat o instanță redis, cu numele redis-db, care rulează pe portul 6379.

3.5 Funcționalități de bază și remarcabile

Pentru a nu mă lungi prea mult, voi menționa și arăta o implementare remarcabilă, care a necesitat mult timp de gândire, răbdare și testare și anume una din diversele modalități folosite, în care datele din MongoDB sunt actualizate. Algoritmul ,,UpdateDocumentAsync”, primește ca parametru id-ul unui curs și o structură de date dinamică. Verifică existența variantei de lucru (draft) a cursului și descompune structura dinamică primită, într-o listă de căi și valori. De exemplu, în cadrul unui curs, zona de detalii are următoarea structură în MongoDB:

```
{
  "courseName": "Introduction to Threads",
  "courseDescription": "This is an introduction to threads, with examples written in C#.",
  "courseDuration": "2 weeks",
  "courseTags": [
    {
      "courseTag": {
        "name": "C#"
      }
    },
    {
      "courseTag": {
        "name": "Thread"
      }
    }
  ]
```

```
}
```

Iar structura primită pentru actualizare este:

courseDetails

```
{
   "courseName": "Name",
   "courseDescription ": "Desctiption",
   "courseTags": [
   {
     "courseTag": {
       "name": "A"
     }
   },
   {
     "courseTag": {
       "name": "B"
     }
   },
   {
     "courseTag": {
       "name": "C"
     }
   },
  ]
}
```

Algoritmul va identifica toate proprietățile ce necesită actualizare, și va extrage urmatoarele căi: courseDetails.courseName cu valoarea ,,Name”, courseDetails.courseDescription cu valoarea ,,Desctiption”, coureDetails.courseTags[0].courseTag cu valoarea ,,A”, courseDetails.courseTags[1].courseTag cu valoarea ,,B” și courseDetails.courseTags[2].courseTag cu valoarea ,,C”, apoi parcurge documentul principal, iar în momentul în care se identifică proprietatea cu ajutorul căilor din listă, valoarea inițială se modifică cu cea de actualizare. În cazul proprietăților de tip listă, se identifică elementul aflat la indexul x, de exemplu courseTags[0], iar dacă există, valoarea se schimbă cu cea

actualizată. Dacă nu există, se adaugă un element nou în listă. La final toate actualizările sunt salvate în documentul curent.

Mai jos voi explica fiecare metodă utilizată pentru a construi algoritmul menționat, și voi incepe cu metoda de bază, care reprezintă o abstracție asupra a ceea ce face algoritmul defapt, și apelează alte metode ajutătoare pentru a obține actualizările necesare:

```
public async Task<DynamicResponse> UpdateDocumentAsync(Guid courseId, dynamic
data)
{
   var draftDocument = await _mongoDbProvider.GetDocumentByCourseIdAsync(courseId,
false);

   if (draftDocument is null)
   {
      return new DynamicResponse(StatusCode.BadRequest, false, "Cannot update live
document");
   }

   List<(string path, string value)> pathsAndValuesOfUpdatedData =
GetPathsAndValuesOfUpdatedData(data);

   UpdateDocumentInternal(draftDocument, pathsAndValuesOfUpdatedData);

   _mongoDbProvider.UpdateEntireDocumentAsync(draftDocument);

   return new DynamicResponse(StatusCode.NoContent, true, "Course document updated
successfully");
}
```

Metoda „GetPathsAndValuesOfUpdatedData(data)” reprezintă una dintre metodele ajutătoare care, se folosește de structura dinamică primită în cerere, și construiește recursiv, o listă de căi de access și valorile actualizate aferente. Transformă datele inițiale într-un dicționar, apoi pentru fiecare pereche cheie-valoare, construiește calea către proprietatea ce necesită actualizare, evitând proprietățile de tip „id”. Dacă valoarea perechii este de tipul

JObject (JSON Object), atunci se transformă într-un dicționar, se parcurge recursiv cu noul dicționar, iar la final de recursie, se salvează într-o listă căile și valorile identificate în urma acesteia. În cazul în care valoarea este de tipul JArray (JSON Array), atunci o transformăm într-o listă de dicționare, daca lista este goală, se salvează cheia și o valoare string.Empty (adică gol), dacă lista conține elemente, se actualizează calea de acces să conțină indexi, iar pentru fiecare element din listă (un dicționar), se reia recursia. La final pentru a evita construcția căilor eronate, de exemplu: courseTag[0][1] din calea curentă se elimină indexul. La finalul tuturor recursiilor, se returnează lista construită. Mai jos voi plasa codul aferent acestui algoritm:

```
private List<(string path, string value)> GetPathsAndValuesOfUpdatedData(dynamic
updatedData, string path = "")
{
    var pathAndValues = new List<(string, string)>();
    var updatedDataDictionary = (IDictionary<string, object>)updatedData;

    foreach (var (key, value) in updatedDataDictionary)
    {
        var currentPath = string.IsNullOrEmpty(path) ? key : $"{path}.{key}";

        if (currentPath.Contains("id"))
        {
            continue;
        }

        switch (value)
        {
            case JObject jObject:
            {
                var dictionary = jObject.ToObject<IDictionary<string, object>>();

                var pathAndValuesFromRecursion =
```

```
GetPathsAndValuesOfUpdatedData(dictionary!, currentPath);
            pathAndValues.AddRange(pathAndValuesFromRecursion);

            continue;
        }

        case JArray jArray:
        {
            var list = jArray.ToObject<List<IDictionary<string, object>>>();

            if (list!.Count == 0)
            {
                pathAndValues.Add((currentPath, ""));
            }

            for (var index = 0; index < list.Count; index++)
            {
                currentPath = $"{currentPath}[{index}]";

                switch (list[index])
                {
                    case { } dictionary:
                    {
                        var pathAndValuesFromRecursion =
GetPathsAndValuesOfUpdatedData(dictionary, currentPath);
                        pathAndValues.AddRange(pathAndValuesFromRecursion);
                        break;
                    }
                }

                currentPath = currentPath.Replace($"[{index}]", "");
            }

            continue;
```

```
            }
        }

        pathAndValues.Add((currentPath, value.ToString())!);
    }

    return pathAndValues;
}
```

Metoda „UpdateDocumentInternal(draftDocument, pathsAndValuesOfUpdatedData)” reprezintă o altă metodă, care efectuează efectiv actualizarea documentului, folosindu-se de lista de căi și valori, precum și documentul inițial. Aceasta din punct de vedere al funcționalității este asemanătoare cu metoda prezentată anterior, pentru creearea căilor și valorilor, dar diferă următoarele: se ține cont de poziția curentă în document, iar dacă există un element în lista primită de căi, pentru care poziția curentă se apropie de calea finală, se continuă recursia în funcție de tipul de date al valorii curente din dicționarul parcurs: alt dicționar sau o listă, doar că în cazul listei, la final de recursie se apelează altă metodă „AddArrayItems(currentPath, pathsWithUpdatedValues, list)” despre care vom vorbi ulterior. La final de recursii, pentru fiecare iterație, se salvează într-o listă cheile din documentul inițial, care reprezintă proprietățile efective ce necesită actualizare. La finalul iterațiilor realizate pe structura documentului inițial, pentru toate cheile salvate în lista menționată anterior, se actualizează valorile documentului. În cazul unei liste, cu valoare goală (cum am menționat în metoda anterioară), se va crea o listă nouă, fără elemente. Totodată, se șterge din documentul inițial toate valorile de tip „recordId” ce reprezintă un identificator temporar, care urmărește modificările unui document înainte de salvare. Mai jos voi prezenta codul:

```
private void UpdateDocumentInternal(dynamic document, IList<(string path, string value)>
pathsWithUpdatedValues, string path = "")
{
    var documentDictionary = (IDictionary<string, object>)document;
```

```
var keysToUpdate = new List<string>();

foreach (var (key, value) in documentDictionary)
{
    var currentPath = string.IsNullOrEmpty(path) ? key : $"{path}.{key}";

    if (currentPath.Contains("id"))
    {
        continue;
    }

    if (pathsWithUpdatedValues.Any(item => item.path.StartsWith(currentPath)))
    {
        switch (value)
        {
            case IDictionary<string, object> dictionary:
            {
                UpdateDocumentInternal(dictionary, pathsWithUpdatedValues, currentPath);
                break;
            }

            case IList<dynamic> list:
            {
                for (var index = 0; index < list.Count; index++)
                {
                    currentPath = $"{currentPath}[{index}]";

                    switch (list[index])
                    {
                        case { } dictionary:
                        {
                            UpdateDocumentInternal(dictionary, pathsWithUpdatedValues,
currentPath);
                            break;
```

```
                }
            }

            currentPath = currentPath.Replace($"[{index}]", "");
        }

        AddArrayItems(currentPath, pathsWithUpdatedValues, list);

        break;
      }
    }

    var updatedValue = pathsWithUpdatedValues.FirstOrDefault(element =>
element.path == currentPath).value;

    if (updatedValue != null)
    {
      keysToUpdate.Add(key);
    }
  }
}

foreach (var key in keysToUpdate)
{
  var currentPath = string.IsNullOrEmpty(path) ? key : $"{path}.{key}";
  var updatedValue = pathsWithUpdatedValues.First(element => element.path ==
currentPath).value;

  if (documentDictionary[key] is IList<dynamic> list)
  {
    if (string.IsNullOrEmpty(updatedValue))
    {
      documentDictionary[key] = new List<dynamic>();
      continue;
```

```
            }

            list.Add(updatedValue);
        }

        documentDictionary[key] = updatedValue;
    }

    documentDictionary.Remove("$recordId");
}
```

Mai sus am menționat metoda „AddArrayItems(currentPath, pathsWithUpdatedValues, list)”, aceasta este responsabilă cu adăugarea de elemente noi într-o listă existentă din cadrul documentului inițial, în situațiile în care structura de actualizare conține mai multe elemente decât cele deja existente. Aceasta este apelată automat la finalul recursiei din metoda de actualizare „UpdateDocumentInternal”, doar în cazul în care proprietatea curentă din document este o listă și trebuie completată cu noi obiecte. Pentru început, se identifică toate căile din lista de actualizări care încep cu calea curentă, urmat de un index de listă (ex: courseTags[0]). Apoi, pentru fiecare astfel de cale, se extrage indexul numeric dintre paranteze și se verifică dacă acel element există deja în listă. Dacă nu există (adică indexul este mai mare decât numărul de elemente din listă), se construiește un obiect nou corespunzător, care urmează să fie adăugat în listă.

În cazul în care structura de adăugat este simplă (fără alte câmpuri în interior), se adaugă valoarea directă. În schimb, dacă structura este compusă (ex: courseTags[3].courseTag.name), se descompune calea rămasă (courseTag.name) în câmpuri și se construiește un obiect imbricat apelând metoda auxiliară „CreateNestedObjectFromFields” pe care o voi prezenta ulterior. Mai jos voi arăta codul pentru metoda curentă:

```
private void AddArrayItems(string currentPath, IList<(string path, string value)>
pathsWithUpdatedValues, IList<dynamic> list)
{
```

```
var arrayItemUpdates = pathsWithUpdatedValues
    .Where(x => x.path.StartsWith($"{currentPath}["))
    .ToList();

foreach (var (updatePath, updateValue) in arrayItemUpdates)
{
    var bracketIndex = updatePath.IndexOf('[');
    var closeBracketIndex = updatePath.IndexOf(']');

    if (bracketIndex < 0 || closeBracketIndex < 0)
    {
        continue;
    }

    var indexString = updatePath.Substring(bracketIndex + 1,
        closeBracketIndex - bracketIndex - 1);

    if (int.TryParse(indexString, out var targetIndex))
    {
        if (targetIndex < list.Count)
        {
            continue;
        }

        var leftoverPath = updatePath[(closeBracketIndex + 1)..];
        leftoverPath = leftoverPath.TrimStart('.');

        dynamic newObj;
        if (string.IsNullOrEmpty(leftoverPath))
        {
            newObj = updateValue;
        }
        else
        {
```

```
            var objectFields = leftoverPath.Split('.');
            newObj = CreateNestedObjectFromFields(objectFields, updateValue);
        }

        if (newObj is IDictionary<string, object> newDict)
        {
            newDict["_id"] = DateTimeOffset.UtcNow.ToUnixTimeSeconds();
        }

        list.Add(newObj);
    }
  }
}
```

Ultima metodă folosită în cadrul acestui algoritm este ,,CreateNestedObjectFromFields” o metodă recursivă care are rolul de a construi un obiect imbricat (nested) pornind de la o listă de câmpuri (fields), rezultată dintr-o cale de tip a.b.c, și o valoare finală care trebuie plasată la nivelul cel mai adânc. De exemplu, dacă metoda primește un array de câmpuri precum ["courseTag", "name"] și valoarea ,,A”, rezultatul final va fi un obiect de forma:

```
{
  "courseTag": {
    "name": "A"
  }
}
```

Această metodă este esențială atunci când trebuie să reconstruim o structură JSON pe baza unei căi textuale. În implementare, dacă lista de câmpuri are doar un singur element, metoda creează un obiect de tip ,,ExpandoObject”, îi adaugă câmpul respectiv cu valoarea primită și îl returnează. În caz contrar, creează un nou obiect ,,ExpandoObject”, și la primul nivel setează câmpul curent ca o cheie, iar ca valoare a acelei chei apelează recursiv aceeași metodă, trecând restul câmpurilor.

Acest comportament recursiv continuă până când se ajunge la ultimul câmp, moment în care obiectul final este complet și poate fi folosit pentru a actualiza structura principală.

Practic, metoda generează dinamic arbori de obiecte JSON, în funcție de cât de adâncă este calea specificată.

```
private dynamic CreateNestedObjectFromFields(string[] fields, string finalValue)
{
   if (fields.Length == 1)
   {
      dynamic leaf = new System.Dynamic.ExpandoObject();
      ((IDictionary<string, object>)leaf)[fields[0]] = finalValue;
      return leaf;
   }

   dynamic root = new System.Dynamic.ExpandoObject();
   ((IDictionary<string, object>)root)[fields[0]] =
      CreateNestedObjectFromFields(fields.Skip(1).ToArray(), finalValue);

   return root;
}
```

3.6 Comunicarea cu serviciul web

Comunicarea dintre partea de frontend (Angular) și backend (ASP.NET Core Web API) este realizată prin intermediul unui set de endpoint-uri REST, expuse în mod standard. Aplicația se bazează pe un sistem de tip client–server, în care frontend-ul consumă datele prin cereri HTTP, folosind metoda ,,HttpClient” din Angular.

În cadrul aplicației, fiecare modul are asociat un fișier de tip ,,.service.ts”, în care sunt definite metodele necesare pentru interacțiunea cu backend-ul. De exemplu, ,,UserService” oferă metode pentru a obține date despre utilizatori (getUserData, getAdministratorUsers, getCandidateUsers) sau pentru a șterge utilizatori (deleteUserById). Aceste metode trimit cereri HTTP de tip GET, POST sau DELETE către rutele definite în controller-ele API-ului backend, construind automat adresele complete ale endpoint-urilor.

Un exemplu de metodă din cadrul ,,UserService” ar fi:

```
getUserData(userId?: string): Observable<UserProfileDto> {
   const endpoint = userId ? `data/${userId}` : 'data';
   const url =
`${this.API_BASE_URL}${UserManagementApiEndpoints.User}${endpoint}`;
   return this.http.get<UserProfileDto>(url);
}
```

Această metodă returnează un „Observable”, utilizat în Angular pentru a gestiona operații asincrone, cum ar fi cererile HTTP, astfel, în loc să returneze direct rezultatul (care va fi disponibil abia după ce serverul răspunde), metoda returnează imediat un „Observable”, iar acesta va emite obiectul „UserProfileDto” atunci când răspunsul HTTP este primit cu succes.

Pentru a obține efectiv datele, componenta care apelează metoda va trebui să se aboneze la acest Observable:

```
private loadUserProfile(): void {
   this.userService.getUserData().subscribe({
      next: (profile) => {
         this.userProfile = profile;
         this.userPermissions = profile.permissions

         const userData = {
            firstName: profile.firstName,
            lastName: profile.lastName,
            email: profile.email,
            userType: profile.userType,
            permissions: profile.permissions,
            id: profile.id
         };

         localStorage.setItem('userProfile', JSON.stringify(userData));
      },
      error: (err) => {
         console.error('Failed to fetch user profile:', err);
```

```
        }
    });
}
```

De asemenea am implementat o funcționalitate de tip spinner, care, pornește înaintea unei cereri HTTP, și se încheie ulterior primirii rezultatului de la backend (doar că am adăugat o mică întârziere, deoarece cererile se procesau foarte rapid, iar spinner-ul nu apuca să se afișeze)

```
this.userService.getAdministratorUsers(searchCommand)
    .pipe(
        delay(300),
        finalize(() => this.spinnerService.hide())
    )
    .subscribe({
    next: (response) => {
        this.users = response.users;
        this.totalUsers = response.count;
        this.updatePagination();
    },
    error: (error) => console.error('Error fetching administrators:', error),
});
```

(prin pipe putem gestiona un observable înainte să ne abonăm la el, de exmplu: filtra, întârzia sau transforma datele).

Pe partea de backend, fiecare endpoint este protejat cu un atribut personalizat numit [WithAuthorization] sau cu cel uzual [Authorize] . Atributul creat de mine implementează interfața ,,IasyncAuthorizationFilter” și verifică existența token-ului în cache-ul distribuit (Redis) și rolul utilizatorului curent. Dacă utilizatorul nu este autentificat sau nu are permisiunile corespunzătoare (de exemplu, pentru acțiuni rezervate administratorilor), cererea este respinsă cu un cod HTTP corespunzător (401 neautorizat sau 403 interzis). De exemplu:

```
[WithAuthorization(IsAdministrator = true)]
[HttpPost("create")]
public async Task<DynamicResponse> CreateCourseAsync([FromBody] dynamic
courseDetails)
{
    var response = await
_courseService.CreateCourseAsync(JsonConvert.SerializeObject(courseDetails));

    return response;
}
```

sau

```
[Authorize]
[HttpGet("sections/{courseId:guid}")]
public async Task<ActionResult<DynamicResponse>> GetCourseSectionsAsync(
    [FromRoute] Guid courseId,
    [FromQuery] int? pageIndex = null)
{
    var response = await _sectionService.GetCourseSectionsAsync(courseId, pageIndex);

    if (!response.IsSuccessful)
    {
        return BadRequest(response);
    }

    return Ok(response);
}
```

În Frontend, fiecare componentă poate să depindă de niște servicii. Aceste servicii conțin funcții care trimit cereri de tip HTTP spre serviciul de Backend, folosind o anumită rută. Pe partea de Backend, controller-ul cu funcționalitatea ce definește ruta la care s-a trimis cererea injectează prin constructor-ul său o interfață a unui serviciu, cu ajutorul căreia se apelează funcționalitatea concretă. La final datele se obțin și se trimit înapoi la Frontend, unde datele

ajung în cadrul componentei și se folosesc pentru a randa anumite elemente de tip HTML, stilizate cu SCSS.

4. Testarea sistemului informatic

Acest capitol este dedicat procesului de testare a aplicației dezvoltate. Fiind vorba de o soluție cu componente multiple, s-a optat pentru testare manuală, acoperind principalele scenarii de utilizare ale aplicației. Se verifică corectitudinea fluxurilor de înregistrare, autentificare, gestionare a utilizatorilor și permisiunilor, accesarea cursurilor și generarea certificărilor. Testarea a vizat atât partea de frontend, cât și interacțiunea cu serviciile backend, pentru a asigura funcționarea corectă a aplicației în ansamblu și pentru a remedia eventualele erori apărute în timpul execuției.

4.1 Testare manuală

Pentru testarea aplicației, s-a adoptat o abordare de tip testare manuală, datorită complexității aplicației și a numărului mare de componente implicate (frontend, backend, fluxuri de autentificare, gestionare date, etc.). Scopul principal al acestei etape a fost verificarea funcționării corecte a fiecărui modul în parte, precum și a fluxului complet de utilizare.

Procesul de testare a început odată cu dezvoltarea backend-ului, unde au fost verificate fiecare rută și funcționalitate prin intermediul interfeței ,,SwaggerUI”. S-au efectuat cereri manuale către API, iar în cazurile în care apăreau erori sau comportamente neașteptate, s-a apelat la depanare linie cu linie folosind mediul de dezvoltare (debug mode) pentru identificarea cauzelor și remedierea acestora. Mai jos voi prezenta interfața Swagger (care pornea o dată cu start-ul serviciului de backend, pentru fiecare microserviciu):

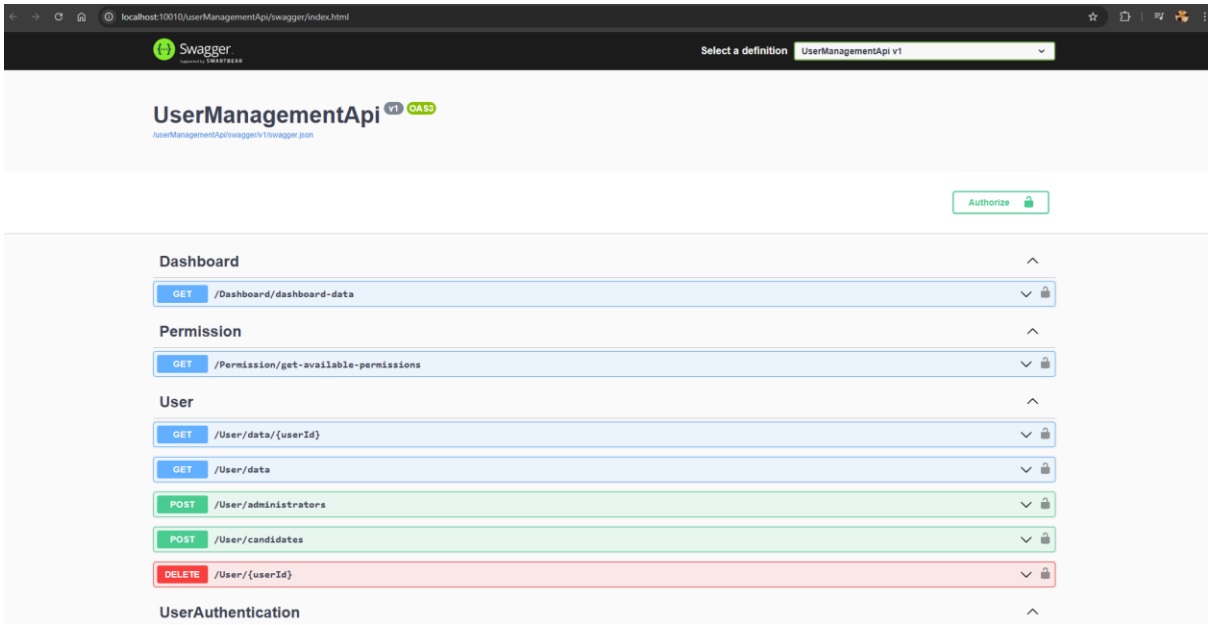


Figura 17. Prezentarea interfeței Swagger pentru UserManagement.Api

Mai sus este prezentată interfața Swagger, în care se pot observa endpoint-urile aplicației, fiecare cu specificul tip de cerere HTTP.

Pentru a lucra cu Swagger, aplicația fiind securizată, primul pas a fost să extrag token-ul de acces din localStorage (de pe partea de Frontend), regăsit în zona „Application” din uneletele de dezvoltare ale browser-ului (dev tools), mai jos voi prezenta modul de acces al acestui token:

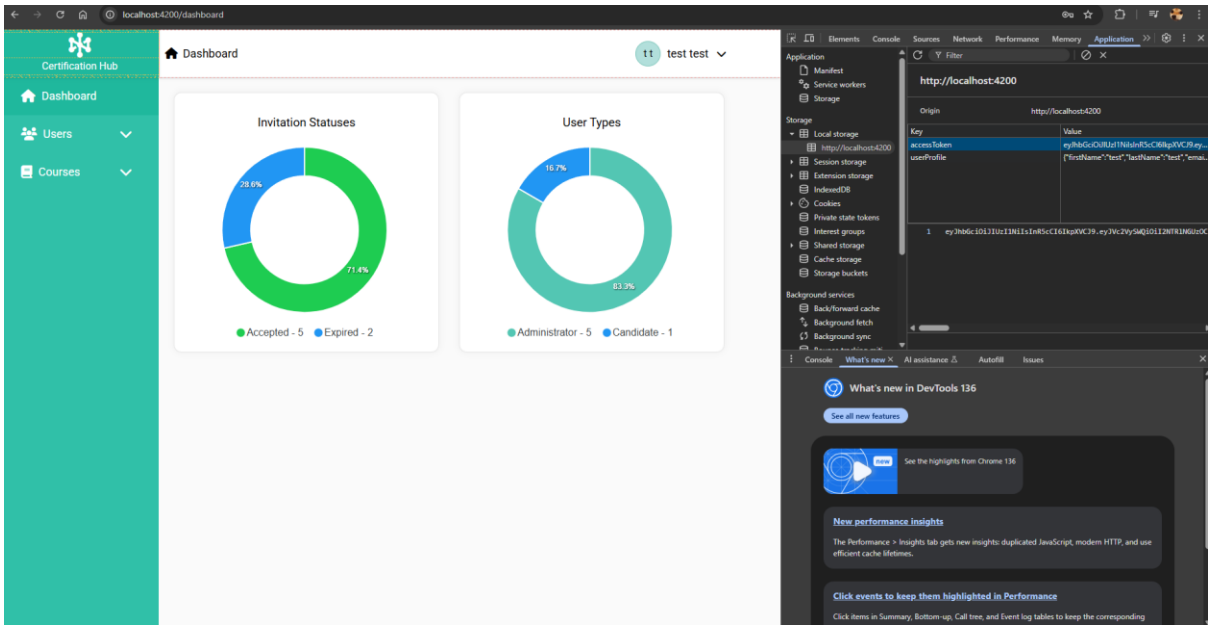


Figura 18. Prezentarea zonei de unde se extrage token-ul de acces pentru cereri

Mai sus se poate observa faptul ca am deschis „dev tools” și am accesat zona „Application” de unde am extras token-ul din „localStorage”.

Următorul pas este de a pune token-ul de acces în zona de „Authorize” din Swagger:

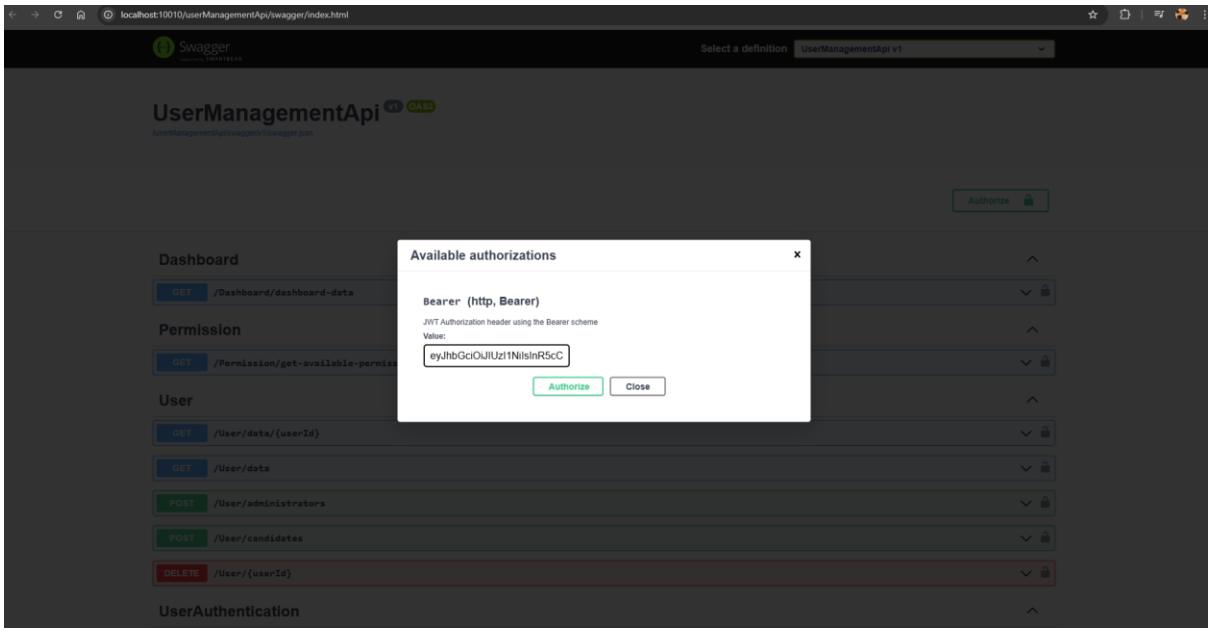


Figura 19. Prezentarea zonei de autentificare din cadrul Swagger UI

Mai sus este prezentată zona de autentificare din zona Swagger, unde trebuie să adaug un token de acces și apoi sa accessez „Authorize”.

Iar ultimul pas este de a face o cerere de tip HTTP, folosind Swagger:

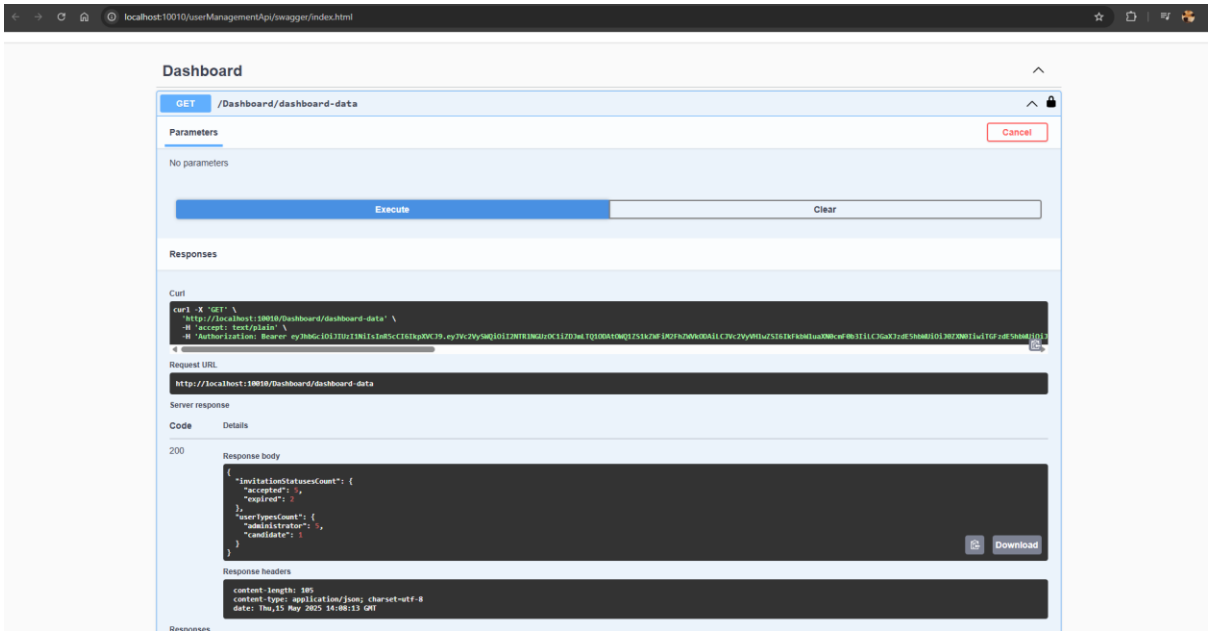


Figura 20. Cerere realizată în cadrul Swagger UI

Mai sus se observă realizarea unei cereri simple, realizată în cadrul Swagger.

De asemenea, aplicația a fost testată și prin rularea serverului în modul debug, efectuând cereri direct din frontend pentru a verifica dacă integrarea între componente funcționează

corect. În această etapă, un rol important l-au avut uneltele de dezvoltare ale browserului, în special secțiunea Network, unde au fost monitorizate cererile HTTP, payload-urile trimise, răspunsurile primite și statusurile acestora.

Pentru testarea anumitor funcționalități protejate, precum cele destinate doar utilizatorilor autentificați, s-a folosit manual token-ul de acces stocat în localStorage-ul aplicației. Acest token a fost copiat și utilizat ulterior în Swagger pentru a simula cereri autentificate, fără a fi necesară logarea de fiecare dată prin frontend.

Această abordare iterativă de tip ,,try-and-fix" a permis validarea comportamentului aplicației în diferite situații, precum și identificarea eventualelor inconsistențe logice sau funcționale. Testarea manuală s-a dovedit eficientă în această etapă, oferind control total asupra fluxurilor de test și permițând ajustări rapide în cazul apariției unor probleme.

Concluzii

În această lucrare am urmărit dezvoltarea unei aplicații web care să simplifice procesul de creare, administrare și livrare a cursurilor online, cu accent pe evaluarea cunoștințelor și generarea certificărilor aferente. Prin intermediul Certification Hub am reușit să ofer o soluție coerentă și funcțională, care răspunde unor nevoi reale din sfera educației moderne, în special accesibilitatea și flexibilitatea procesului de învățare.

Utilitatea practică a aplicației este semnificativă, atât pentru cadrele didactice sau formatorii care doresc să gestioneze conținutul educațional într-un mod structurat și eficient, cât și pentru cursanții care pot accesa gratuit materiale valoroase, se pot autoevalua și pot obține certificări relevante. Aplicația reduce astfel barierele financiare și tehnice care pot apărea în cadrul învățării online, promovând un model educațional deschis și inclusiv.

Dezvoltarea aplicației mi-a oferit nu doar ocazia de a pune în practică cunoștințele acumulate în domeniul ingineriei software, ci și o perspectivă mai clară asupra importanței accesului liber la educație. Mulțumesc cadrului didactic coordonator pentru sprijinul oferit, precum și tuturor celor care au contribuit indirect prin sugestii sau feedback pe parcursul dezvoltării aplicației.

Bibliografie

Adeel , E., Mohammed, A. M., Cagatay , C., & Deepti , M. (2022, 04 26). RESTful API Testing Methodologies: Rationale, Challenges, and Solution Directions. *Applied Sciences, 12*(9), 4369. Preluat pe 05 10, 2025, de pe https://doi.org/10.3390/app12094369

Anders , H., Scott , W., & Peter , G. (2003, 10 01). *C# Language Specification.* Boston, MA, United States: Wesley Longman Publishing Co., Inc. Retrieved 05 10, 2025, from https://dl.acm.org/doi/abs/10.5555/861332

BEZERRA, L. N., & Terra da SILVA, M. (2017). A review of literature on the reasons that cause the high dropout rates in the MOOCs. *Revista ESPACIOS, 38*(5), 11. Preluat pe 05 10, 2025, de pe https://www.revistaespacios.com/a17v38n05/17380511.html

Blinowski, G., Ojdowska, A., & Przybyłek, A. (2022, 10). Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation. *IEEE Access, 10*, 20357-20374. Preluat pe 05 10, 2025, de pe https://doi.org/10.1109/ACCESS.2022.3152803

Cincovic, J., Delcev, S., & Draskovic, D. (2019). Architecture of web applications based on Angular Framework: A Case Study. (pg. 254-259). Belgrade: University of Belgrade, School of Electrical Engineering. Preluat pe 05 10, 2025, de pe https://www.eventiotic.com/eventiotic/files/Papers/URL/df6b5054-816e-4bee-b983-663fb87be2cd.pdf

Győrödi, C., Sotoc, R., & Győrödi, R. (2015, 11). A Comparative Study of Relational and Non-Relational Database Models in a Web- Based Application. *International Journal of Advanced Computer Science and Applications, 6*(11), 78-83. Preluat pe 05 10, 2025, de pe https://www.researchgate.net/publication/285752873_A_Comparative_Study_of_Relational_and_Non-Relational_Database_Models_in_a_Web-_Based_Application

Jayaraman, K. D., & Siddharth, E. (2025, 05). Harnessing the Power of Entity Framework Core for Scalable Database Solutions. *Journal of Quantum Science and Technology (JQST), 2*(1), 151-171. Preluat pe 05 10, 2025, de pe ResearchGate: https://www.researchgate.net/publication/390200510_Harnessing_the_Power_of_Entity_Framework_Core_for_Scalable_Database_Solutions

Juba, S., Vannahme, A., & Volkov, A. (2015). *Learning PostgreSQL.* Birmingham, United Kingdom: Packt Publishing. Preluat pe 05 10, 2025, de pe https://books.google.ro/books?hl=ro&lr=&id=jfKoCwAAQBAJ&oi=fnd&pg=PP1&dq=What+is+PostgreSQL&ots=n3I4mnm4RJ&sig=2A7cwq8GBlQUJgkn7uQ8B19Gmps&redir_esc=y#v=onepage&q&f=false

Sulyman, S. H. (2014, 02). Client-Server Model . *IOSR Journal of Computer Engineering (IOSR-JCE) , 16*(1), 67-71. Preluat pe 05 10, 2025, de pe https://www.researchgate.net/publication/271295146_Client-Server_Model

***Daniel , R., Rick , A., & Shaun , L. (2025, 04 21). *Overview of ASP.NET Core.* Preluat pe 05 10, 2025, de pe Microsoft Learn: https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core?view=aspnetcore-9.0

***Docker Inc. (fără an). *What is Docker?* Preluat pe 05 10, 2025, de pe Docker Docs: https://docs.docker.com/get-started/docker-overview/#the-docker-platform

***GeeksforGeeks. (2025). *Client-Server Model.* Preluat pe 05 10, 2025, de pe GeeksforGeeks: https://www.geeksforgeeks.org/system-design/client-server-model/

***Microsoft. (2022, 12 10). *An introduction to NuGet.* Preluat pe 05 10, 2025, de pe Microsoft Learn: https://learn.microsoft.com/en-us/nuget/what-is-nuget

***Microsoft. (2023, 07 03). *Common web application architectures.* Preluat pe 05 10, 2025, de pe Microsoft Learn Documentation: https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures

***Microsoft. (2023, 07 3). *Common web application architectures.* Preluat pe 05 10, 2025, de pe Microsoft Learn: https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures

***Redis Ltd. (fără an). *Introduction to Redis.* Preluat pe 05 10, 2025 , de pe Redis About: https://redis.io/about/

***Sass. (fără an). *Sass Basics.* Preluat pe 05 10, 2025, de pe Sass: https://sass-lang.com/guide/